

Product Requirements Document (PRD)

AI-Powered Learning-Path Recommender for Employee Skill-Gap Analysis and Personalized Development

Version: 1.0 **Document Type:** Complete Implementation-Ready PRD **Project Type:** College/Research Project **Target Audience:** Development Team, Academic Advisors, Research Reviewers

Table of Contents

1. Product Vision
2. Problem Definition
3. Product Goals and Objectives
4. Target Users
5. Core User Journey
6. Functional Requirements
7. Skill-Gap Analysis Engine
8. Evidence-Based Skill Profiling
9. Skill Knowledge Graph
10. Hybrid Recommendation Engine
11. Course Recommendation
12. Project Recommendation
13. Mentor Recommendation
14. Prerequisite-Aware Learning Path Generation
15. Learning Path Optimization
16. Adaptive Learning
17. Assessment Engine
18. GitHub Skill Verification
19. Explainable AI
20. AI Learning Assistant
21. What-If Career Simulation
22. Skill Digital Twin

- 23. System Architecture
 - 24. Database Design
 - 25. API Design
 - 26. Frontend Design
 - 27. Recommendation Explainability UI
 - 28. Dataset Design
 - 29. Machine Learning Methodology
 - 30. Evaluation Framework
 - 31. Research Contribution
 - 32. Security and Privacy
 - 33. Responsible AI
 - 34. Non-Functional Requirements
 - 35. Technology Stack
 - 36. Project Folder Structure
 - 37. Development Roadmap
 - 38. MVP vs Advanced Version
 - 39. Example End-to-End Scenario
 - 40. Final Product Definition
 - 41. What Makes This Project Unique?
-

1. Product Vision

1.1 Product Name

SkillPath AI — *An Intelligent Learning-Path Recommender with Evidence-Based Skill Profiling and Prerequisite-Aware Optimization*

Alternative names: **SkillWeaver**, **PathForge AI**, **CompetenSee**

1.2 One-Line Description

An AI-powered system that transforms an employee's evidence-backed skill profile into a personalized, prerequisite-aware, and continuously adaptive learning path comprising courses, projects, mentors, and assessments — with full explainability.

1.3 Problem Statement

Employees and organizations lack a systematic, evidence-based mechanism to:

- Precisely identify which skills an employee lacks relative to a target role,
- Understand the *prerequisite structure* among those skills,
- Receive a *sequenced* learning path (not a bag of courses),
- Verify skill acquisition through assessments and evidence,
- Adapt the path as the employee progresses.

Existing platforms (Coursera, Udemy, LinkedIn Learning, Pluralsight) recommend courses based on popularity, collaborative filtering, or superficial keyword matching. They do **not**:

- Model prerequisite dependencies between skills,
- Aggregate multi-source evidence into a confidence-weighted skill profile,
- Optimize learning sequences under time and dependency constraints,
- Provide explainable, evidence-grounded recommendations.

1.4 Current Limitations of Existing Platforms

Limitation	Description
Popularity-driven	Recommend what is popular, not what the employee <i>needs</i>
No prerequisite reasoning	Recommend advanced courses before foundational ones
Static paths	Learning paths don't adapt to assessment results
Self-reported skills	Rely on unverified user-declared proficiency
No project/mentor integration	Courses alone don't build practical competence
Opaque recommendations	No explanation for why a resource was recommended
No role-competency mapping	Weak connection between skills and target roles
No evidence aggregation	Ignore GitHub, projects, assessments as evidence

1.5 Proposed Solution

SkillPath AI is a multi-component intelligent system that:

1. **Extracts skills** from resumes, job descriptions, and GitHub using NLP/NER and sentence transformers.
2. **Builds an evidence-based skill profile** by aggregating signals from multiple sources with confidence weighting.
3. **Computes skill gaps** relative to target roles using a weighted gap model (proficiency gap × importance × role criticality × confidence).
4. **Constructs a skill knowledge graph** encoding prerequisites, relatedness, and resource mappings.

5. **Generates a prerequisite-respecting learning path** using graph algorithms + constraint optimization (OR-Tools).
6. **Recommends courses, projects, and mentors** via a hybrid engine (content + semantic + collaborative + KG + gap relevance).
7. **Adapts the path** based on assessment results and progress.
8. **Explains every recommendation** through a deterministic explainability layer, optionally verbalized by an LLM.
9. **Provides an AI learning assistant** grounded in the learner's profile via RAG.

1.6 Product Vision

To become the most technically rigorous, explainable, and evidence-grounded learning-path recommendation system that connects *skills* → *courses* → *projects* → *mentors* → *assessments* → *evidence* in a single adaptive loop, setting a new standard for competency-driven workforce development.

1.7 Product Mission

To empower every employee with a **transparent, personalized, and verifiable** development roadmap that closes the gap between their current competencies and their target role — while giving organizations actionable, evidence-based workforce intelligence.

1.8 Target Users

- **Employees/Learners** — seeking skill development or role transitions
- **Managers** — seeking team skill-gap visibility
- **Mentors** — offering guidance
- **HR/L&D Administrators** — managing workforce development
- **System Administrators** — managing the platform

1.9 Primary Use Cases

1. Employee uploads resume → receives skill profile → selects target role → gets optimized learning path.
2. Manager views team skill-gap heatmap → identifies training priorities.
3. Employee completes assessment → path adapts automatically.
4. Employee explores "What-If" scenarios across multiple target roles.
5. Employee asks the AI assistant "What should I learn next?"

1.10 Key Differentiators

1. **Evidence-based skill profiling** with confidence scores (not self-reported).
2. **Knowledge-graph-driven prerequisite reasoning** for path ordering.
3. **Constraint-based learning-path optimization** (OR-Tools).
4. **Hybrid recommendation** combining 6 signals.
5. **Explainable recommendations** grounded in deterministic logic.
6. **Continuous adaptation** via assessment loop.
7. **Integration of projects and mentors**, not just courses.

1.11 Why This Is More Than a Recommendation System

A conventional recommendation system answers: *"What course should I take?"*

SkillPath AI answers:

- *"What skills do I actually have, and how confident are we?"*
- *"What skills do I lack for my target role, and how important are they?"*
- *"In what ORDER should I learn them, respecting prerequisites?"*
- *"Which course, project, AND mentor together will build this skill?"*
- *"How do I verify I've actually learned it?"*
- *"Why was this recommended, and how does it fit my path?"*

It is a **competency development system** with recommendation as one component.

1.12 Why This Is Academically Interesting

- Combines **NLP, knowledge graphs, hybrid recommendation, constraint optimization, and explainable AI** in one system.
- Introduces a **novel evidence-weighted skill confidence model**.
- Formulates **learning-path generation as a constrained optimization problem**.
- Provides a **research framework for evaluating prerequisite-aware recommendations**.
- Bridges **EdTech, HR Tech, and applied ML** research.

2. Problem Definition

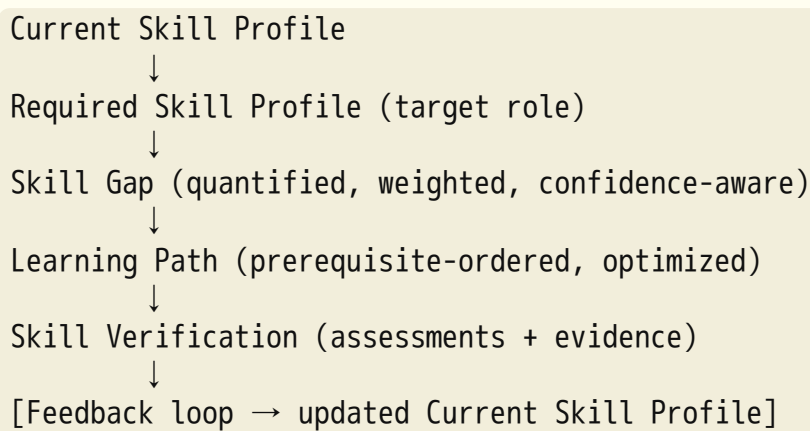
2.1 Real-World Problem

Organizations invest billions in employee training, yet:

- **Employees don't know exactly which skills they lack.** Self-assessment is biased; managers lack granular visibility.

- **Mapping skills to roles is ad hoc.** Job descriptions are unstructured; competency frameworks are static.
- **Course overload causes decision paralysis.** Thousands of courses exist; relevance is unclear.
- **Prerequisites are ignored.** Learners enroll in advanced courses without foundations, fail, and disengage.
- **Hands-on practice is missing.** Courses alone don't build competence; projects do.
- **Mentors are underutilized.** Matching is informal and inefficient.
- **Learning paths are static.** They don't adapt when a learner struggles or accelerates.
- **Self-reported skills are unreliable.** Overconfidence and underconfidence are common.
- **Recommendations are black-box.** Learners distrust opaque suggestions.
- **Skills aren't connected to careers.** Learners can't see how learning maps to role progression.

2.2 The Core Gap Chain



The gap: No existing system closes this entire loop with evidence, prerequisites, optimization, and explainability.

2.3 Problem Formalization

Given:

- Employee e with skill profile $S_e = \{(s_i, \ell_i, c_i)\}$ where ℓ_i = proficiency level, c_i = confidence
- Target role r with required skills $R_r = \{(s_j, \ell_j^{req}, w_j)\}$ where w_j = importance weight
- Resource pool $\mathcal{R}$ = courses $\cup$ projects $\cup$ mentors
- Prerequisite graph $G = (V, E)$ where V = skills, E = prerequisite edges
- Constraints: time budget T , availability A , deadline D

Find:

- A learning path $P = [x_1, x_2, \dots, x_n]$ where each $x_k \in \mathcal{R}$
- Such that P maximizes skill coverage of gaps, respects prerequisites, minimizes time, and is explainable.

3. Product Goals and Objectives

3.1 Primary Objectives

ID	Objective	Success Metric
P1	Accurately extract skills from resumes/JDs	F1 $\geq$ 0.85 on skill extraction
P2	Compute evidence-based skill profiles	Confidence calibration error $\leq$ 0.10
P3	Identify skill gaps relative to roles	Gap ranking NDCG@10 $\geq$ 0.80
P4	Generate prerequisite-valid learning paths	Prerequisite violation rate = 0%
P5	Recommend relevant courses/projects/mentors	Precision@10 $\geq$ 0.70
P6	Adapt paths based on assessments	Path update latency < 5s

3.2 Secondary Objectives

- Provide manager dashboards for team skill visibility.
- Enable "What-If" career exploration.
- Support mentor matching with explainability.
- Offer an AI assistant for learning support.

3.3 Technical Objectives

- Modular microservice architecture (FastAPI).
- Hybrid recommendation with ≥ 6 signals.
- Knowledge graph with ≥ 8 relationship types.
- Constraint optimization using OR-Tools.
- Explainability layer for every recommendation.
- Vector search for semantic similarity.

3.4 Research Objectives

- Propose a novel **evidence-weighted skill confidence model**.
- Formulate **learning-path generation as constrained optimization**.
- Design a **hybrid recommendation framework** for competency development.
- Publish evaluation comparing 5 recommendation approaches.
- Contribute an **explainable recommendation taxonomy** for EdTech.

3.5 User Experience Objectives

- Onboarding in < 5 minutes.
- Learning path visualization that is intuitive.
- Every recommendation has a "Why?" button.
- Mobile-responsive UI.
- Accessibility (WCAG 2.1 AA).

3.6 Measurable Success Criteria

Criterion	Target
Skill extraction F1	≥ 0.85
Recommendation Precision@10	≥ 0.70
NDCG@10	≥ 0.80
Prerequisite violations	0
Path generation time	< 3s
User satisfaction (SUS)	≥ 75
Explainability satisfaction	≥ 80% find explanations useful
System uptime (demo)	≥ 99%

4. Target Users

4.1 Employee / Learner

Aspect	Detail
Goals	Improve skills; transition to new role; track progress
Pain Points	Don't know gaps; overwhelmed by courses; no clear order
Needs	Personalized path; evidence-based profile; adaptive updates
Interactions	Upload resume; select role; view path; take assessments; chat with assistant
Permissions	Read/write own profile; read resources; submit assessments

4.2 Manager

Aspect	Detail
Goals	Understand team skill gaps; plan training; allocate budget
Pain Points	No granular visibility; can't prioritize training
Needs	Team skill heatmap; gap reports; development recommendations
Interactions	View team dashboard; export reports; assign learning paths
Permissions	Read team profiles (aggregated); assign paths; view reports

4.3 Mentor

Aspect	Detail
Goals	Guide learners; share expertise; track mentee progress
Pain Points	Poor matching; unclear expectations; time management
Needs	Matched mentees; session scheduling; progress visibility
Interactions	View matched learners; schedule sessions; provide feedback
Permissions	Read mentee profiles (shared); write feedback; manage availability

4.4 HR / L&D Administrator

Aspect	Detail
Goals	Workforce development; compliance training; ROI tracking
Pain Points	Fragmented data; no skill taxonomy; hard to measure impact
Needs	Org-wide skill analytics; role competency frameworks; reports
Interactions	Manage roles/skills; view analytics; configure programs
Permissions	Read all profiles; manage taxonomy; configure system

4.5 System Administrator

Aspect	Detail
Goals	Maintain platform; manage content; ensure security
Pain Points	Data quality; integration complexity; scaling
Needs	Content management; user management; monitoring
Interactions	CRUD on all entities; configure ML models; monitor logs

Aspect	Detail
Permissions	Full system access

5. Core User Journey

STEP 1: Employee Registers

- Email/password or SSO
- Role selection (Employee)



STEP 2: Upload Resume / Provide Profile

- PDF/DOCX upload or manual entry
- Education, experience, self-declared skills



STEP 3: System Extracts Skills

- NLP pipeline: text extraction → cleaning → NER → normalization → taxonomy mapping → proficiency estimation
- Output: Initial skill profile with confidence scores



STEP 4: Employee Selects Target Role

- Browse role catalog or paste job description
- System extracts required skills from JD



STEP 5: System Identifies Required Skills

- Map JD skills to internal taxonomy
- Retrieve role competency profile
- Assign importance weights



STEP 6: Current Skills Assessed

- Optional: take diagnostic assessments
- Aggregate evidence (resume, GitHub, assessments)
- Compute confidence-weighted proficiency



STEP 7: Skill Gaps Calculated

- $\text{Gap}(\text{skill}) = f(\text{required}, \text{current}, \text{importance}, \text{confidence})$
- Rank gaps by priority



STEP 8: Prerequisite Graph Constructed

- Load skill subgraph for gap skills
- Identify prerequisite chains
- Detect missing foundations



STEP 9: Resources Retrieved

- Courses matching gap skills
- Projects for practice
- Mentors with expertise



STEP 10: Hybrid Recommendation Engine Ranks Resources

- 6-signal scoring: semantic + gap + prerequisite + difficulty + preference + collaborative



STEP 11: Learning Path Optimizer Generates Sequence

- Topological sort respecting prerequisites
- Constraint optimization (time, availability, deadline)
- Output: Ordered path with milestones



STEP 12: Employee Follows Learning Path

- View path visualization
- Access courses, projects, mentors
- Track progress



STEP 13: Assessments Measure Progress

- MCQs, coding, conceptual, practical
- Update skill proficiency + confidence



STEP 14: Skill Profile Updated

- New evidence added
- Proficiency recalculated
- Confidence updated



STEP 15: Learning Path Adapts

- Remove mastered skills
- Add remedial content if assessment failed
- Re-optimize sequence
- Notify employee

Step-by-Step Explanation

Step	Explanation
1	Basic authentication; role determines permissions
2	Resume is primary evidence source; manual entry for gaps
3	NLP pipeline extracts and normalizes skills; confidence reflects extraction certainty
4	Target role anchors the entire gap analysis
5	JD parsing handles unstructured requirements; taxonomy mapping ensures consistency
6	Diagnostic assessments optional but improve confidence
7	Gap score combines multiple factors, not just difference
8	Prerequisites determine path ordering; missing foundations are inserted
9	Multi-resource retrieval ensures courses + projects + mentors
10	Hybrid scoring balances relevance, difficulty, and personalization
11	Optimization respects constraints (time, deadline) and dependencies
12	UI provides visibility and motivation
13	Assessments provide verified evidence
14	Profile is dynamic, not static
15	Adaptation is the key differentiator vs. static platforms

6. Functional Requirements

6.1 Employee Profile

FR-1.1 The system shall allow employees to create a profile with:

- Full name, email, password
- Current job title, department, years of experience
- Education (degree, institution, year)
- Work experience (company, role, duration, description)
- Self-declared skills with proficiency (1–5 scale)
- Target role(s) selection
- Available learning hours per week
- Learning preferences (video, text, hands-on, etc.)

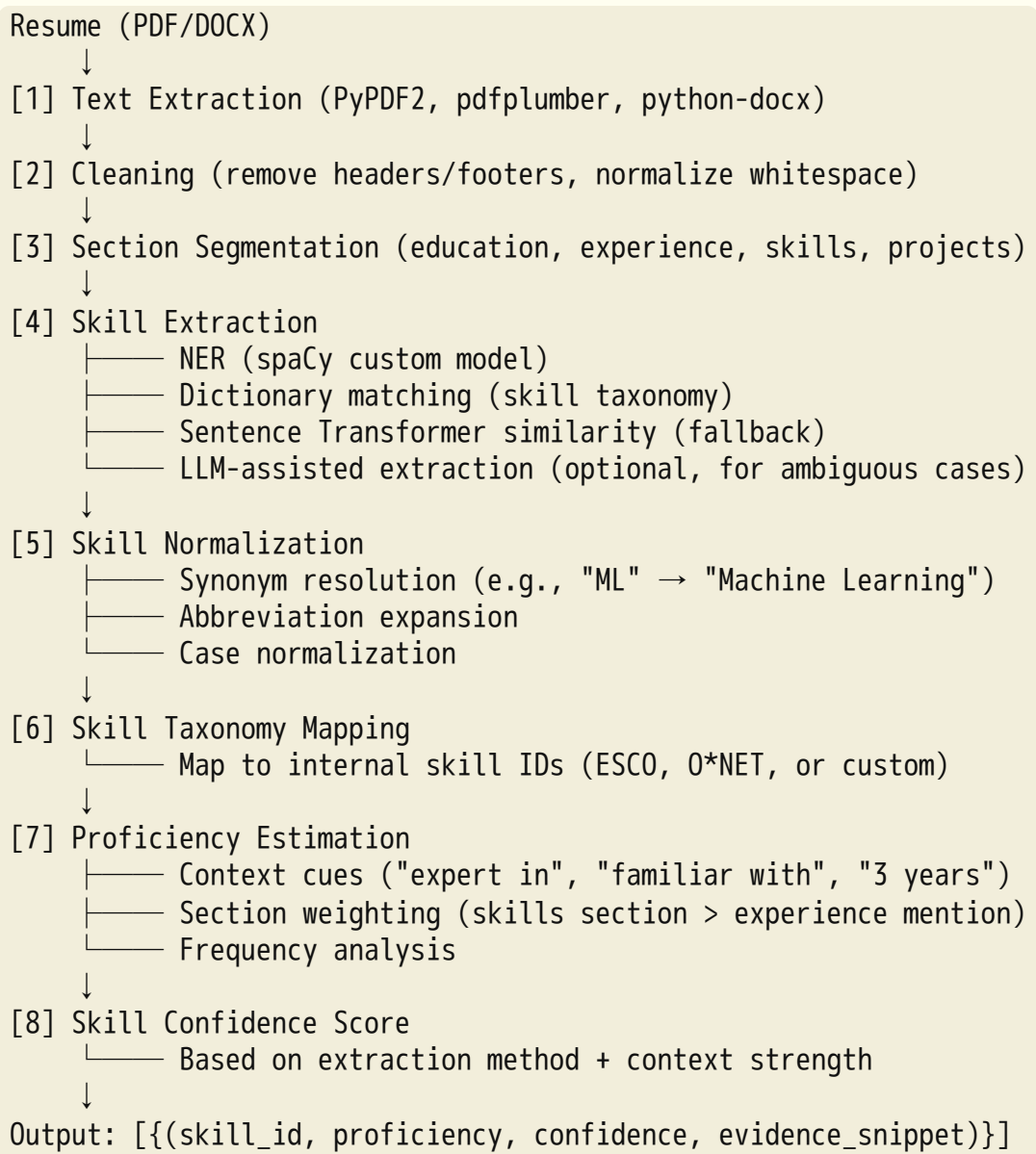
FR-1.2 The system shall support resume upload (PDF, DOCX, TXT).

FR-1.3 The system shall allow profile editing and version history.

FR-1.4 The system shall compute a profile completeness score.

6.2 Resume Skill Extraction

NLP Pipeline



Approaches

Approach	Use Case	Pros	Cons
Dictionary + Regex	Known skills	Fast, precise	Misses novel skills
NER (spaCy)	Entity extraction	Context-aware	Needs training data
Sentence Transformers	Semantic matching	Handles variations	Compute-heavy
LLM-assisted	Ambiguous/novel skills	Flexible	Cost, hallucination risk

Recommended: Hybrid — dictionary + NER primary, sentence transformers for fallback, LLM for edge cases.

6.3 Job Description Analysis

FR-3.1 The system shall accept JD input (paste or upload).

FR-3.2 The system shall extract:

- Required skills (must-have)
- Preferred skills (nice-to-have)
- Proficiency requirements (if stated)
- Technologies/tools
- Years of experience
- Role category

FR-3.3 The system shall map extracted skills to internal taxonomy.

FR-3.4 The system shall assign importance weights:

- Required = 1.0
- Preferred = 0.6
- Mentioned = 0.4

FR-3.5 The system shall handle missing skills by inferring from role category.

7. Skill-Gap Analysis Engine

7.1 Basic Gap Model

$$\text{Gap}_{\text{basic}}(s) = L_{\text{req}}(s) - L_{\text{cur}}(s)$$

Where:

- $L_{\text{req}}(s)$ = required proficiency level (1–5)
- $L_{\text{cur}}(s)$ = current proficiency level (1–5)

7.2 Enhanced Gap Model

$$\text{Gap}(s) = \max(0, L_{\text{req}}(s) - L_{\text{cur}}(s)) \times I(s) \times C(s) \times R(s) \times E(s)$$

Where:

- $I(s)$ = importance weight (0–1) from role
- $C(s)$ = confidence in current proficiency (0–1)
- $R(s)$ = role criticality (0–1)
- $E(s)$ = evidence strength (0–1)

7.3 Component Definitions

Component	Definition	Range
L_{req}	Required level from role/JD	1–5
L_{cur}	Current level from evidence	1–5
$I(s)$	Importance in target role	0–1
$C(s)$	Confidence in current level	0–1
$R(s)$	Criticality (role-specific)	0–1
$E(s)$	Evidence strength	0–1

7.4 Example Calculations

Example 1: Python

Factor	Value
L_{req}	4
L_{cur}	3.5
I	0.95
C	0.90
R	0.85
E	0.88

$$\text{Gap} = (4 - 3.5) \times 0.95 \times 0.90 \times 0.85 \times 0.88 = 0.5 \times 0.639 = 0.32$$

Example 2: RAG

Factor	Value
L_{req}	4
L_{cur}	0
I	0.90
C	0.95
R	0.90
E	0.92

$$\text{Gap} = 4 \times 0.90 \times 0.95 \times 0.90 \times 0.92 = 4 \times 0.708 = 2.83$$

Interpretation: RAG has a much higher gap score than Python, so it should be prioritized.

7.5 Gap Priority Ranking

Skills are ranked by $\text{Gap}(s)$ descending. Ties broken by importance.

7.6 Gap Categories

Category	Condition
Critical	$\text{Gap} > 2.0$
High	$1.0 < \text{Gap} \leq 2.0$
Medium	$0.5 < \text{Gap} \leq 1.0$
Low	$0 < \text{Gap} \leq 0.5$
None	$\text{Gap} = 0$

8. Evidence-Based Skill Profiling

8.1 Evidence Sources

Source	Type	Reliability	Example
Resume	Text	Medium	"3 years Python experience"
GitHub	Code	High	Repos with Python code
Completed Courses	Structured	Medium	Certificate from Coursera
Assessment Scores	Structured	High	87% on Python test
Projects	Structured	High	Built ML pipeline
Certifications	Structured	High	AWS ML Specialty
Work Experience	Text	Medium	"Led data team using Python"
Self-declaration	Manual	Low	"I know Python"

8.2 Evidence Model

```
Evidence {
  id: UUID
  employee_id: UUID
  skill_id: UUID
  source_type: Enum(RESUME, GITHUB, COURSE, ASSESSMENT, PROJECT, CERT, WORK, SELF)
  source_id: UUID (reference to source record)
  raw_text: String (optional)
  extracted_level: Float (0-5)
  reliability: Float (0-1) // source reliability
  recency: Float (0-1) // how recent
  strength: Float (0-1) // evidence strength
  timestamp: DateTime
}
```

8.3 Skill Confidence Score

For a skill s with evidence set $\mathcal{E}_s$:

$$C(s) = 1 - \prod_{e \in \mathcal{E}_s} (1 - r_e \cdot \sigma_e \cdot \rho_e)$$

Where:

- r_e = reliability of evidence e
- σ_e = strength of evidence e
- ρ_e = recency factor of evidence e

This models confidence as "1 minus the probability that all evidence is wrong."

8.4 Proficiency Aggregation

$$L_{\text{cur}}(s) = \frac{\sum_{e \in \mathcal{E}_s} w_e \cdot \ell_e}{\sum_{e \in \mathcal{E}_s} w_e}$$

Where $w_e = r_e \cdot \sigma_e \cdot \rho_e$ and ℓ_e = level indicated by evidence.

8.5 Example: Python

Source	Level	Reliability	Strength	Recency	Weight
Resume	4.0	0.6	0.7	0.9	0.378
GitHub	4.5	0.95	0.9	0.8	0.684
Assessment	4.2	0.95	1.0	0.95	0.9025
Projects	4.3	0.85	0.9	0.85	0.650

$$L_{\text{cur}} = \frac{0.378 \cdot 4.0 + 0.684 \cdot 4.5 + 0.9025 \cdot 4.2 + 0.650 \cdot 4.3}{0.378 + 0.684 + 0.9025 + 0.650} = \frac{1.512 + 3.078 + 3.791 + 2.795}{2.6145} = \frac{11.176}{2.6145} = 4.27$$

$$C(s) = 1 - (1 - 0.378)(1 - 0.684)(1 - 0.9025)(1 - 0.650) = 1 - (0.622)(0.316)(0.0975)(0.350) = 1 - 0.0067 = 0.993$$

8.6 Conflicting Evidence Handling

When evidence conflicts (e.g., resume says expert, assessment says beginner):

- 1. **Weight by reliability:** Higher-reliability sources dominate.
- 2. **Flag for verification:** If variance > threshold, prompt for assessment.
- 3. **Recency bias:** Recent evidence weighted higher.
- 4. **Transparency:** Show all evidence in UI with conflicts highlighted.

Variance = $\frac{\sum w_e (\ell_e - L_{cur})^2}{\sum w_e}$

If Variance > 0.5, flag as "conflicting evidence – assessment recommended."

8.7 Evidence Strength Scoring

Source	Strength Formula
Assessment	$\sigma = \text{score}/100$
GitHub	$\sigma = f(\text{commits}, \text{complexity}, \text{recency})$
Project	$\sigma = f(\text{complexity}, \text{relevance}, \text{completion})$
Course	$\sigma = f(\text{grade}, \text{completion}, \text{provider})$
Resume	$\sigma = f(\text{context cues}, \text{specificity})$
Self	$\sigma = 0.3 \text{ (low)}$

9. Skill Knowledge Graph

9.1 Graph Schema

Node Types

Node	Properties
Employee	id, name, role
Skill	id, name, category, description
Course	id, title, provider, duration, difficulty
Project	id, title, difficulty, duration
Mentor	id, name, expertise
TargetRole	id, title, category

Node	Properties
Assessment	id, type, skill
Certification	id, name, provider

Relationship Types

Relationship	From → To	Properties
REQUIRES	TargetRole → Skill	importance, required_level
PREREQUISITE_OF	Skill → Skill	strength (0–1)
RELATED_TO	Skill → Skill	similarity (0–1)
TEACHES	Course → Skill	proficiency_gain, coverage
PRACTICES	Project → Skill	proficiency_gain, difficulty
EXPERT_IN	Mentor → Skill	level, years
RECOMMENDED_FOR	Course/Project → TargetRole	relevance
PART_OF	Skill → SkillCategory	—
COMPLEMENTS	Skill → Skill	—
HAS_SKILL	Employee → Skill	level, confidence
COMPLETED	Employee → Course	date, grade
MENTORS	Mentor → Employee	status

9.2 Example Graph Structures

Prerequisite Chain:

Statistics —PREREQUISITE_OF→ Machine Learning
Machine Learning —PREREQUISITE_OF→ Deep Learning
Deep Learning —PREREQUISITE_OF→ Transformers
Transformers —PREREQUISITE_OF→ LLMs
LLMs —PREREQUISITE_OF→ RAG
RAG —PREREQUISITE_OF→ Vector Databases

Course-Skill Mapping:

Course: "Deep Learning Specialization"
|—— TEACHES → Neural Networks (level 4)
|—— TEACHES → CNN (level 3)
|—— TEACHES → RNN (level 3)
|—— REQUIRES → Machine Learning (prerequisite)

Mentor Expertise:

Mentor: Dr. Smith

EXPERT_IN → RAG (level 5, 3 years)

EXPERT_IN → LangChain (level 5, 2 years)

EXPERT_IN → Vector Databases (level 4, 3 years)

EXPERT_IN → Python (level 5, 10 years)

9.3 Graph Queries

Find prerequisites for a skill:

cypher

```
MATCH path = (s:Skill {name: 'RAG'})<-[:PREREQUISITE_OF*]-(pre:Skill)
RETURN pre.name, length(path) AS depth
ORDER BY depth
```

Find courses teaching a skill:

cypher

```
MATCH (c:Course)-[:TEACHES]->(s:Skill {name: 'Deep Learning'})
RETURN c.title, c.duration, c.difficulty
```

Find mentors for gap skills:

cypher

```
MATCH (m:Mentor)-[:EXPERT_IN]->(s:Skill)
WHERE s.name IN $gapSkills
RETURN m.name, collect(s.name) AS expertise, count(s) AS overlap
ORDER BY overlap DESC
```

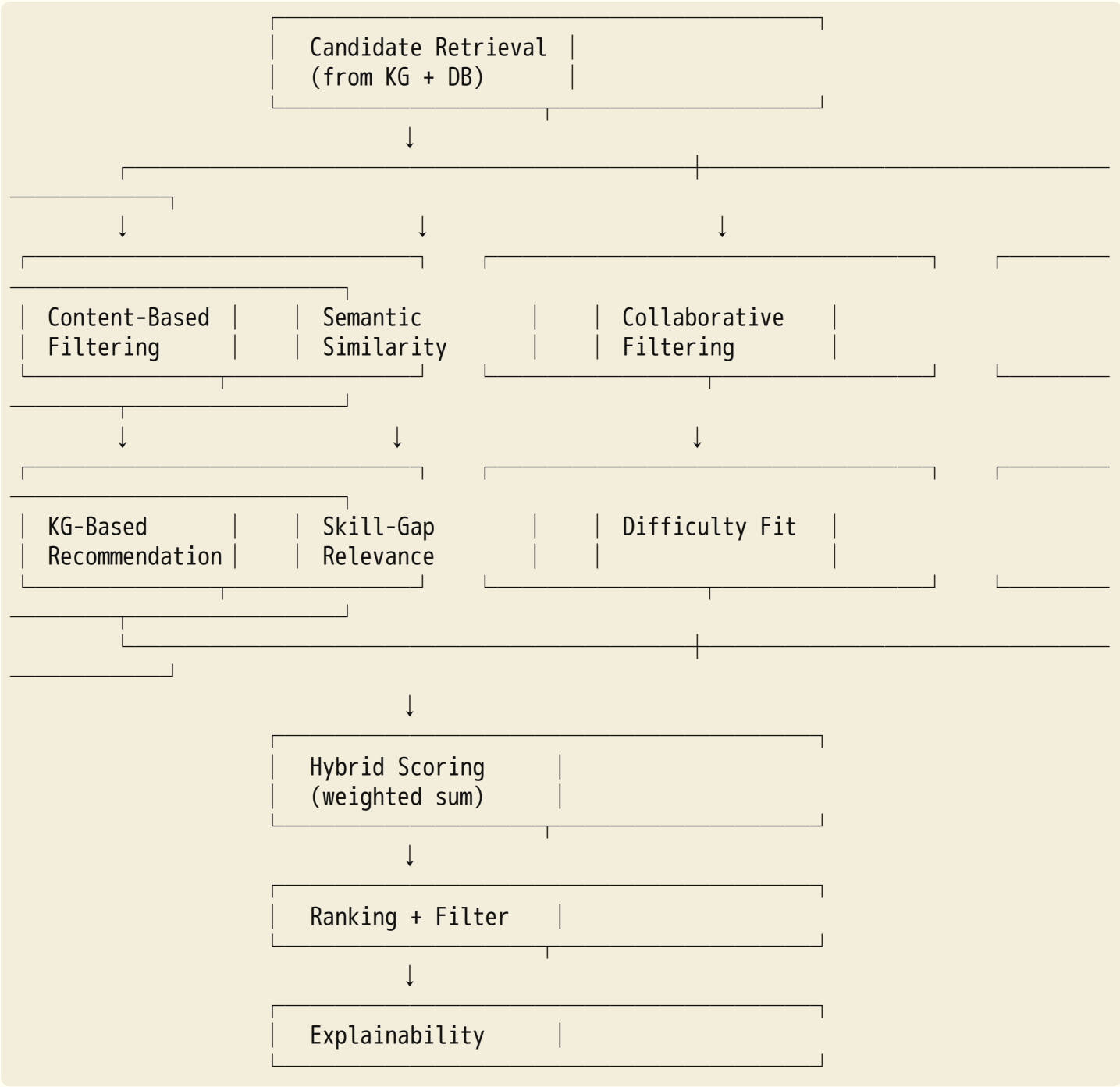
9.4 Technology Recommendation

Option	Pros	Cons	Verdict
Neo4j	Native graph, Cypher, ACID, visualization	Separate DB, licensing	✓ Recommended
NetworkX	Python-native, easy	In-memory, no persistence	For prototyping
PostgreSQL (recursive CTEs)	Single DB, familiar	Slow for deep traversals	Fallback

Recommendation: Neo4j for production graph; NetworkX for algorithm prototyping; PostgreSQL for transactional data.

10. Hybrid Recommendation Engine

10.1 Architecture



10.2 Component 1: Content-Based Filtering

Idea: Match resource features to user profile.

$$\text{Score}_{\text{CB}}(u, r) = \cos(\vec{f}_u, \vec{f}_r)$$

Where $\vec{f}_u$ = user feature vector (skills, preferences), $\vec{f}_r$ = resource feature vector.

10.3 Component 2: Semantic Similarity

Idea: Use sentence embeddings to match skill descriptions to resource descriptions.

$$\text{Score}_{\text{Sem}}(u, r) = \cos(\text{Emb}(\text{gap_desc}_u), \text{Emb}(\text{res_desc}_r))$$

Model: `all-MiniLM-L6-v2` or `all-mpnet-base-v2`.

10.4 Component 3: Collaborative Filtering

Idea: Users with similar profiles who liked resource $r \rightarrow$ recommend to target user.

$$\text{Score}_{\text{CF}}(u, r) = \frac{\sum_{v \in N(u)} \text{sim}(u, v) \cdot 1[v \text{ interacted with } r]}{\sum_{v \in N(u)} \text{sim}(u, v)}$$

Use matrix factorization (ALS) or implicit feedback models.

10.5 Component 4: Knowledge Graph Recommendation

Idea: Traverse graph to find resources connected to gap skills.

$$\text{Score}_{\text{KG}}(u, r) = \sum_{s \in \text{gaps}(u)} \text{path_strength}(s, r)$$

Where path strength = product of edge weights along shortest path.

10.6 Component 5: Skill-Gap Relevance

Idea: Resources addressing high-priority gaps score higher.

$$\text{Score}_{\text{Gap}}(u, r) = \frac{\sum_{s \in \text{skills}(r)} \text{Gap}(s) \cdot \text{coverage}(r, s)}{\sum_{s \in \text{skills}(r)} \text{coverage}(r, s)}$$

10.7 Component 6: Difficulty Fit

Idea: Resource difficulty should match learner's current level + 1 (zone of proximal development).

$$\text{Score}_{\text{Diff}}(u, r) = \exp\left(-\frac{(d_r - (L_{\text{cur}} + 1))^2}{2\sigma^2}\right)$$

10.8 Component 7: User Preference

Idea: Match resource type/format to user preferences.

$$\text{Score}_{\text{Pref}}(u, r) = \text{preference_match}(u.\text{prefs}, r.\text{format})$$

10.9 Hybrid Scoring Formula

$$\text{Score}(u, r) = w_1 \cdot \text{Sem} + w_2 \cdot \text{Gap} + w_3 \cdot \text{Prereq} + w_4 \cdot \text{Diff} + w_5 \cdot \text{Pref} + w_6 \cdot \text{CF} + w_7 \cdot \text{KG}$$

Default weights:

Weight	Component	Value
w_1	Semantic	0.20
w_2	Skill-Gap Relevance	0.30
w_3	Prerequisite Fit	0.15
w_4	Difficulty Fit	0.10
w_5	User Preference	0.10
w_6	Collaborative	0.10
w_7	Knowledge Graph	0.05

10.10 Weight Determination

- 1. **Initial:** Set by domain experts.
- 2. **Tuning:** Grid search / Bayesian optimization on validation set.
- 3. **Learning:** Learn-to-rank (LambdaMART) with user feedback.
- 4. **Adaptive:** Adjust based on user engagement.

10.11 Prerequisite Fit

$$\text{Score}_{\text{Prereq}}(u, r) = \frac{|\{s \in \text{prereq}(r) : L_{\text{cur}}(s) \geq L_{\text{req}}(s)\}|}{|\text{prereq}(r)|}$$

Resources with satisfied prerequisites score higher.

11. Course Recommendation

11.1 Features Used

Feature	Source	Weight
Skill gap addressed	Gap engine	High
Prerequisites satisfied	KG	High
Difficulty match	Course metadata	Medium
Duration fit	Course metadata + availability	Medium
Learning preference	User profile	Medium
Course quality	Ratings, completion rate	Medium
Semantic similarity	Embeddings	Medium

Feature	Source	Weight
Target role relevance	Role mapping	High
Employee progress	History	Low

11.2 Example Recommendation

Employee: Software Engineer, 2 years, Python/SQL/Git, target ML Engineer.

Gap: RAG (Gap = 2.83), Vector Databases (Gap = 2.5), LLMs (Gap = 2.1).

Recommended Courses:

Ran k	Course	Scor e	Why
1	RAG Fundamentals	0.92	Addresses top gap; prerequisites (LLMs) partial ly met
2	Vector Databases with Pinec one	0.88	Addresses #2 gap; no prerequisites
3	LLM Engineering	0.85	Addresses #3 gap; builds foundation for RAG
4	LangChain for Developers	0.82	Complements RAG; high semantic match
5	MLOps Fundamentals	0.75	Role relevance; medium gap

12. Project Recommendation

12.1 Rationale

Courses teach concepts; projects build competence. Every skill gap should have at least one project recommendation.

12.2 Project Selection Criteria

Criterion	Description
Skill coverage	Project practices gap skills
Difficulty	Matches learner's level + 1
Duration	Fits available time
Prerequisites	Learner has required foundations
Relevance	Related to target role

Criterion	Description
Portfolio value	Demonstrates competence
Uniqueness	Not overused

12.3 Example: RAG Skill Gap

Level	Project	Duration	Skills Practiced
Beginner	Build a document Q&A system	1 week	RAG, embeddings, vector search
Intermediate	Build a multi-document RAG with citations	2 weeks	RAG, LangChain, prompt engineering
Advanced	Production-ready enterprise RAG system	4 weeks	RAG, MLOps, scaling, evaluation

12.4 Project Recommendation Algorithm

$$\text{Score}_{\text{Proj}}(u, p) = w_1 \cdot \text{coverage}(p, \text{gaps}) + w_2 \cdot \text{difficulty_fit} + w_3 \cdot \text{prereq_fit} + w_4 \cdot \text{portfolio_value}$$

13. Mentor Recommendation

13.1 Matching Factors

Factor	Weight	Description
Skill expertise overlap	0.30	Mentor's skills $\cap$ gap skills
Domain expertise	0.15	Industry/domain match
Experience level	0.10	Years in field
Availability	0.15	Free slots per month
Learning objectives	0.10	Alignment with mentee goals
Mentoring history	0.10	Ratings, past mentees
Workload	0.10	Current mentee count

13.2 Matching Algorithm

$$\text{Score}_{\text{Mentor}}(u, m) = \sum_i w_i \cdot \text{factor}_i(u, m)$$

Skill overlap: $\text{overlap}(u, m) = \frac{|\text{gaps}(u) \cap \text{expertise}(m)|}{|\text{gaps}(u)|}$

13.3 Explainable Mentor Recommendation

Example output:

Mentor: Dr. Sarah Chen — Match Score: 0.91

Why recommended:

- Expert in **RAG** (5/5), **LangChain** (5/5), **Vector Databases** (4/5) — these overlap with **3 of your top 5 skill gaps**.
- 8 years of industry experience in NLP/GenAI.
- Available for 4 sessions/month (you requested 2–4).
- Previous mentees rated 4.8/5 for clarity and guidance.
- **Recommended focus:** RAG architecture, production deployment, evaluation strategies.

14. Prerequisite-Aware Learning Path Generation

14.1 Core Innovation

The system does **not** recommend independent courses. It constructs an **ordered path** where each step is a prerequisite for the next.

14.2 Prerequisite Graph Construction

For each gap skill s_{gap} :

1. Retrieve all prerequisites from KG: $\text{Pre}(s_{\text{gap}})$
2. Check which prerequisites the learner already satisfies
3. Identify **missing prerequisites** = $\text{Pre}(s_{\text{gap}}) \setminus \text{satisfied}(u)$
4. Recursively expand missing prerequisites
5. Build the full prerequisite DAG

14.3 Example Path

Target: ML Engineer **Gaps:** RAG, Vector DBs, LLMs, Transformers, Deep Learning

Prerequisite expansion:

RAG requires LLMs
LLMs requires Transformers
Transformers requires Deep Learning
Deep Learning requires Machine Learning
Machine Learning requires Statistics + Python

Learner has: Python , Statistics  (partial), ML 

Generated path:

1. Statistics (refresher)
2. Machine Learning
3. Deep Learning
4. Transformers
5. LLMs
6. RAG
7. Vector Databases
8. MLOps
9. Capstone Project

14.4 Ordering Algorithm

Topological Sort with Priority:

python

```
def generate_path(gaps, prereq_graph, resources):
    # 1. Expand gaps with missing prerequisites
    required_skills = expand_prerequisites(gaps, prereq_graph, user)

    # 2. Topological sort
    ordered_skills = topological_sort(prereq_graph, required_skills)

    # 3. For each skill, select best resource
    path = []
    for skill in ordered_skills:
        resource = select_best_resource(skill, resources, user)
        path.append(resource)

    # 4. Optimize sequence (see Section 15)
    path = optimize_sequence(path, constraints)

    return path
```

14.5 Handling Cycles

If the graph has cycles (rare but possible), break cycles by removing the weakest edge (lowest prerequisite strength).

15. Learning Path Optimization

15.1 Optimization Objective

Minimize: $\alpha \cdot \text{TotalTime} + \beta \cdot \text{PrereqViolations} + \gamma \cdot \text{RedundantLearning}$

Maximize: $\delta \cdot \text{SkillCoverage} + \epsilon \cdot \text{RoleRelevance} + \zeta \cdot \text{Progression}$

15.2 Mathematical Formulation

Sets:

- S = skills to learn
- R = resources (courses, projects)
- T = time slots

Decision Variables:

- $x_{r,t} \in \{0, 1\}$ = resource r scheduled at time t
- $y_s \in \{0, 1\}$ = skill s covered

Objective: $\max \sum_{s \in S} w_s y_s - \lambda \sum_{r,t} c_r x_{r,t} - \mu \sum_{(s,s') \in \text{Prereq}} \text{violation}(s, s')$

Constraints:

1. Each resource scheduled at most once: $\sum_t x_{r,t} \leq 1$
2. Time capacity: $\sum_r c_r x_{r,t} \leq \text{available}_t$
3. Prerequisite ordering: $\text{end}(s) \leq \text{start}(s')$ if s prereq of s'
4. Skill coverage: $y_s \leq \sum_{r \text{ teaches } s} \sum_t x_{r,t}$
5. Deadline: $\text{end}(r) \leq D$

15.3 Tools

Tool	Use
OR-Tools CP-SAT	Constraint programming
PuLP	Linear programming
NetworkX	Graph algorithms
Custom heuristics	Greedy + local search

Recommendation: OR-Tools CP-SAT for flexibility.

15.4 Example Optimization

Input:

- Gaps: ML, DL, NLP (total 120 hours needed)
- Available: 10 hours/week
- Deadline: 16 weeks
- Prerequisites: ML $\rightarrow$ DL $\rightarrow$ NLP

Output:

Week 1–4: ML Course (40h)
 Week 5–9: DL Course (50h)
 Week 10–12: NLP Course (30h)
 Week 13–16: Capstone Project (40h) [optional, if time]

15.5 Trade-off Analysis

Scenario	Time	Coverage	Violations
Fastest	12 weeks	70%	0
Balanced	16 weeks	90%	0
Comprehensive	20 weeks	100%	0

System presents options; learner chooses.

16. Adaptive Learning

16.1 Triggers for Adaptation

Trigger	Action
Assessment score < 60%	Insert remedial content
Assessment score > 90%	Skip to advanced content
Course completed	Mark skill as improved
Project completed	Add project evidence
User feedback ("too easy")	Increase difficulty
User feedback ("too hard")	Decrease difficulty
Time constraints changed	Re-optimize sequence

16.2 Skill State Update

After assessment:

$$L_{\text{new}}(s) = \alpha \cdot L_{\text{old}}(s) + (1 - \alpha) \cdot L_{\text{assessment}}(s)$$

Where α = learning rate (e.g., 0.3).

$$C_{\text{new}}(s) = 1 - (1 - C_{\text{old}}(s))(1 - C_{\text{assessment}})$$

16.3 Example Adaptation

Initial path:

After ML assessment (score: 55%):

(Remedial modules inserted)

After DL assessment (score: 95%):

(Skipped basic NLP; added advanced topics)

16.4 Re-optimization Frequency

- After each assessment
- Weekly (if progress data available)
- On user request
- On profile change

17. Assessment Engine

17.1 Assessment Types

Type	Purpose	Auto-graded	Skill Signal
MCQ	Concept check	Yes	Medium
Coding	Practical skill	Yes (test cases)	High
Conceptual	Deep understanding	LLM-assisted	Medium
Practical task	Applied skill	Rubric/peer	High
Project evaluation	Comprehensive	Rubric	Very High

17.2 Assessment-to-Skill Mapping

Each question maps to:

- Skill ID
- Proficiency level tested
- Difficulty
- Weight

17.3 Score Calculation

$$\text{Score}(s) = \frac{\sum_{q \in Q_s} w_q \cdot \text{correct}(q)}{\sum_{q \in Q_s} w_q}$$

17.4 Proficiency Update

Score	Proficiency Level
0–20%	1 (Beginner)
21–40%	2 (Basic)
41–60%	3 (Intermediate)
61–80%	4 (Advanced)
81–100%	5 (Expert)

17.5 Adaptive Testing

Use Item Response Theory (IRT) to select questions:

- Start with medium difficulty
- Correct → harder question
- Incorrect → easier question
- Stop when confidence interval is narrow

$$P(\text{correct}|\theta, b) = \frac{1}{1 + e^{-a(\theta - b)}}$$

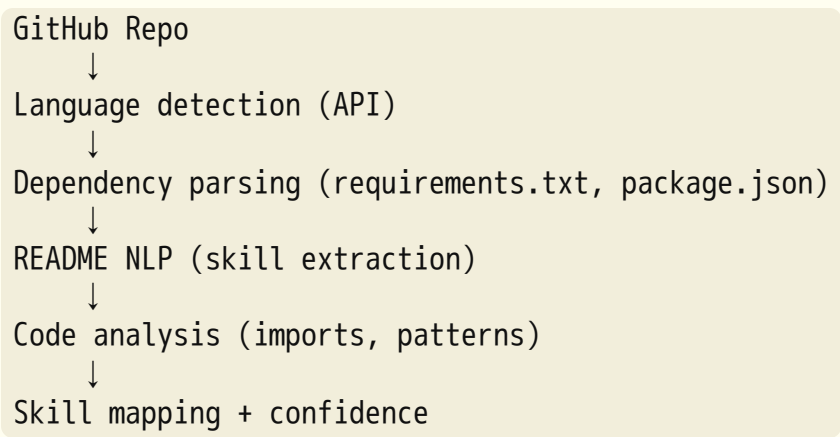
Where θ = ability, b = difficulty, a = discrimination.

18. GitHub Skill Verification

18.1 Data Collected

Data	Source	Skill Signal
Repositories	GitHub API	Project evidence
Languages	Repo metadata	Programming skills
Frameworks	Dependency files	Framework skills
README content	NLP	Skill mentions
Code structure	AST analysis	Complexity
Commit history	API	Activity, consistency
Stars/forks	API	Quality signal

18.2 Skill Mapping



18.3 Example

Repo: rag-chatbot

Signal	Detected Skill	Confidence
Language: Python	Python	0.95
langchain in requirements	LangChain	0.90
pinecone-client in requirements	Vector Databases	0.85
README mentions "RAG"	RAG	0.80
openai import	LLM APIs	0.85

Aggregated: RAG proficiency = 3.5, confidence = 0.85

18.4 Limitations

Limitation	Mitigation
Forked repos	Check if original author
Tutorial code	Analyze complexity, commit history
Outdated repos	Recency weighting
Private repos	Optional OAuth scope
False positives	Require multiple signals

18.5 False Positive Handling

- Require ≥ 2 signals for high confidence
- Check commit authorship

- Analyze code complexity (not just imports)
- Flag suspicious patterns (e.g., 100 repos in 1 day)

19. Explainable AI

19.1 Explainability Principles

1. **Every recommendation has a reason.**
2. **Reasons are grounded in data**, not LLM hallucination.
3. **Explanations are actionable** ("take this because...").
4. **Confidence is communicated.**
5. **Alternatives are shown.**

19.2 Explanation Types

Question	Explanation Source
Why this course?	Score breakdown + gap mapping
Why this project?	Skill coverage + difficulty fit
Why this mentor?	Expertise overlap + availability
Why this order?	Prerequisite chain
Why is this a gap?	Required vs. current + evidence
Why this score?	Weighted components

19.3 Explanation Generation

Deterministic layer:

```
python
```

```
def explain_recommendation(user, resource, scores):
    reasons = []
    if scores['gap'] > 0.7:
        reasons.append(f"Addresses high-priority gap: {top_gap}")
    if scores['prereq'] == 1.0:
        reasons.append("All prerequisites satisfied")
    if scores['diff'] > 0.8:
        reasons.append("Difficulty matches your level")
    if scores['cf'] > 0.6:
        reasons.append("Similar learners found this useful")
```

LLM layer (optional):

Given these deterministic reasons: [...]
 Generate a natural-language explanation for the learner.
 Do NOT add reasons not in the list.

19.4 Example Explanation

Course: RAG Fundamentals Match Score: 92%

Why this course?

- ✓ Addresses your **#1 skill gap: RAG** (gap score: 2.83)
- ✓ Prerequisites satisfied: You have **LLM basics** and **Python**
- ✓ Difficulty matches your level (Intermediate)
- ✓ Duration fits your 10 hrs/week availability (4 weeks)
- ✓ 87% of similar learners completed this course
- ⚠ **Note:** You may want to strengthen **Vector Databases** first (prerequisite strength: 0.6)

Skills you'll gain: RAG architecture, embeddings, retrieval, prompt engineering **Estimated effort:** 40 hours **Next in path:** Vector Databases with Pinecone

20. AI Learning Assistant

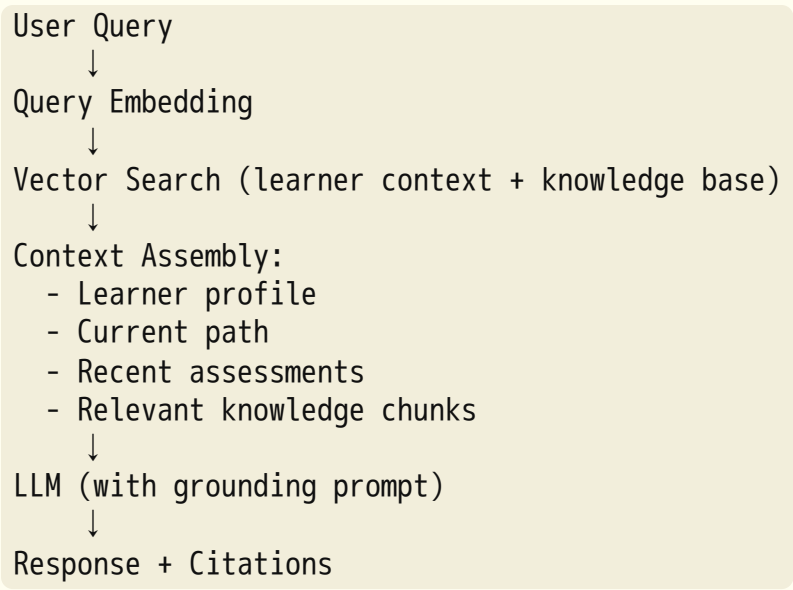
20.1 Purpose

A conversational assistant grounded in the learner's profile, path, and progress.

20.2 Capabilities

Query Type	Example	Response Source
Concept explanation	"Explain attention mechanisms"	LLM + knowledge base
Path reasoning	"Why do I need statistics before ML?"	Prerequisite graph
Project suggestion	"Give me a project for RAG"	Project recommender
Next step	"What should I learn next?"	Learning path
Remediation	"I failed this assessment. What should I revise?"	Assessment + gaps
Progress	"How am I doing?"	Progress tracker

20.3 RAG Architecture



20.4 Context Components

Context	Source	Size
Learner profile	DB	~500 tokens
Current path	DB	~300 tokens
Recent assessments	DB	~200 tokens
Knowledge chunks	Vector DB	~1500 tokens
Conversation history	Session	~500 tokens

20.5 Example Interaction

User: "Why do I need statistics before ML?"

Assistant: "Great question! In your learning path, **Statistics** is a prerequisite for **Machine Learning** because:

- 1. **Core concepts:** ML algorithms like linear regression, logistic regression, and Naive Bayes are built on statistical foundations (distributions, hypothesis testing, MLE).
- 2. **Evaluation:** Understanding statistical significance helps you evaluate model performance correctly (p-values, confidence intervals).
- 3. **Your current level:** You have partial statistics knowledge (level 2.5/5, confidence 0.6). Strengthening it will make ML concepts much easier.
- 4. **Recommended:** The "Statistics for ML" module (12 hours) covers exactly what you need. Would you like to start it?"

21. What-If Career Simulation

21.1 Purpose

Allow employees to compare target roles and see the development effort required for each.

21.2 Comparison Dimensions

Dimension	Description
Required skills	Skills needed for role
Current coverage	% of skills already have
Skill gaps	Missing skills with scores
Learning resources	Courses + projects needed
Estimated effort	Total hours
Suggested projects	Portfolio builders
Suggested mentors	Experts in gap areas
Time to readiness	Based on availability

21.3 Example Comparison

Current Profile: Software Engineer, 2 years, Python/SQL/Git/Basic ML

Target Role	Coverage	Top Gaps	Effort	Time (10h/wk)
ML Engineer	45%	DL, MLOps, Transformers	320h	32 weeks
NLP Engineer	40%	Transformers, NLP, LLMs	350h	35 weeks
Data Scientist	60%	Statistics, Visualization, SQL Advanced	200h	20 weeks
GenAI Engineer	35%	LLMs, RAG, Vector DBs, Prompt Eng	380h	38 weeks

21.4 Disclaimer

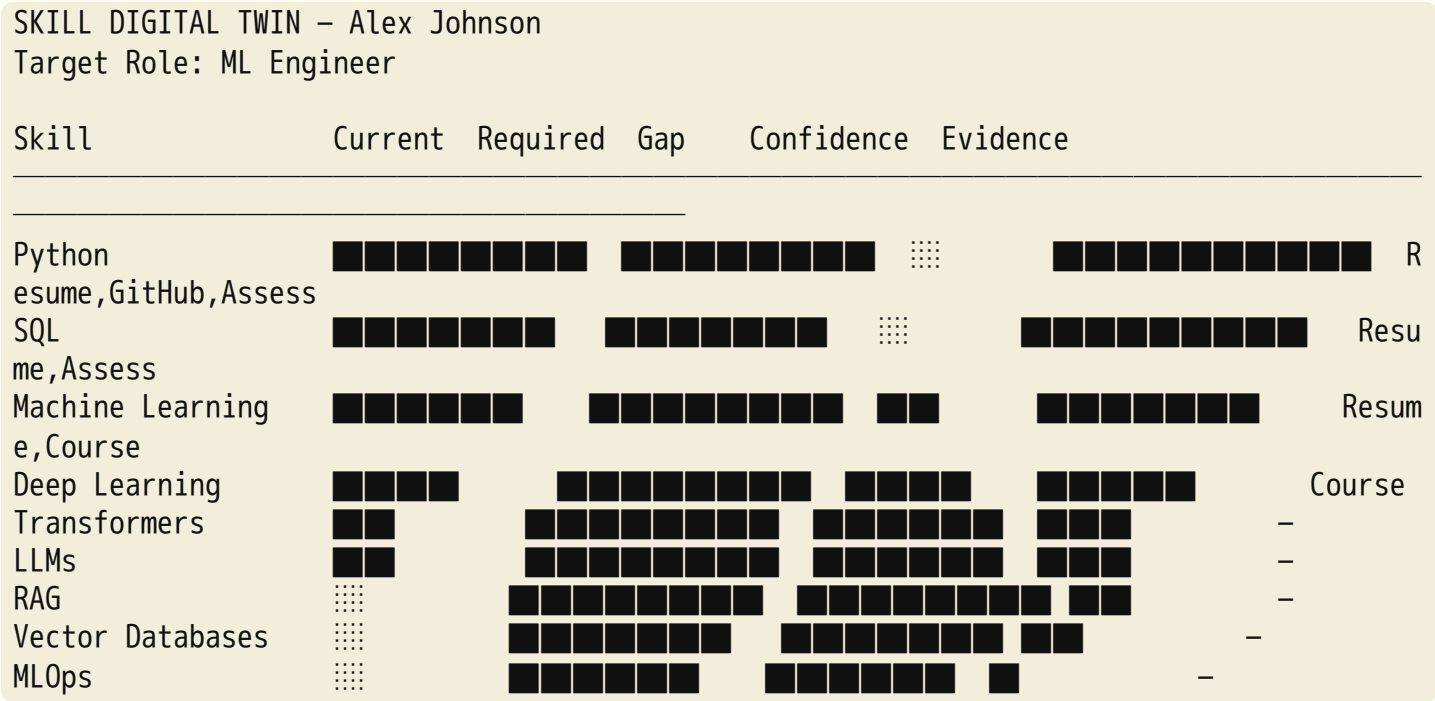
This is a competency-development simulation, not a hiring prediction. It shows what skills you would need to develop for each role, based on current market competency frameworks. It does not predict employment outcomes.

22. Skill Digital Twin

22.1 Concept

A visual, dynamic representation of the employee's skill profile.

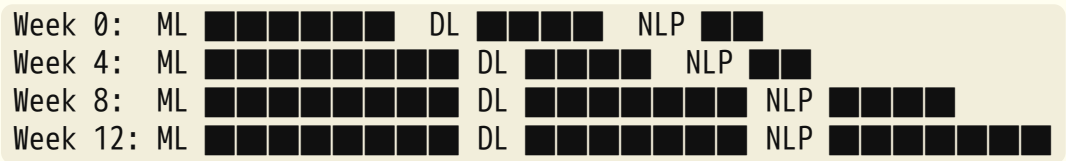
22.2 Visualization



22.3 Features

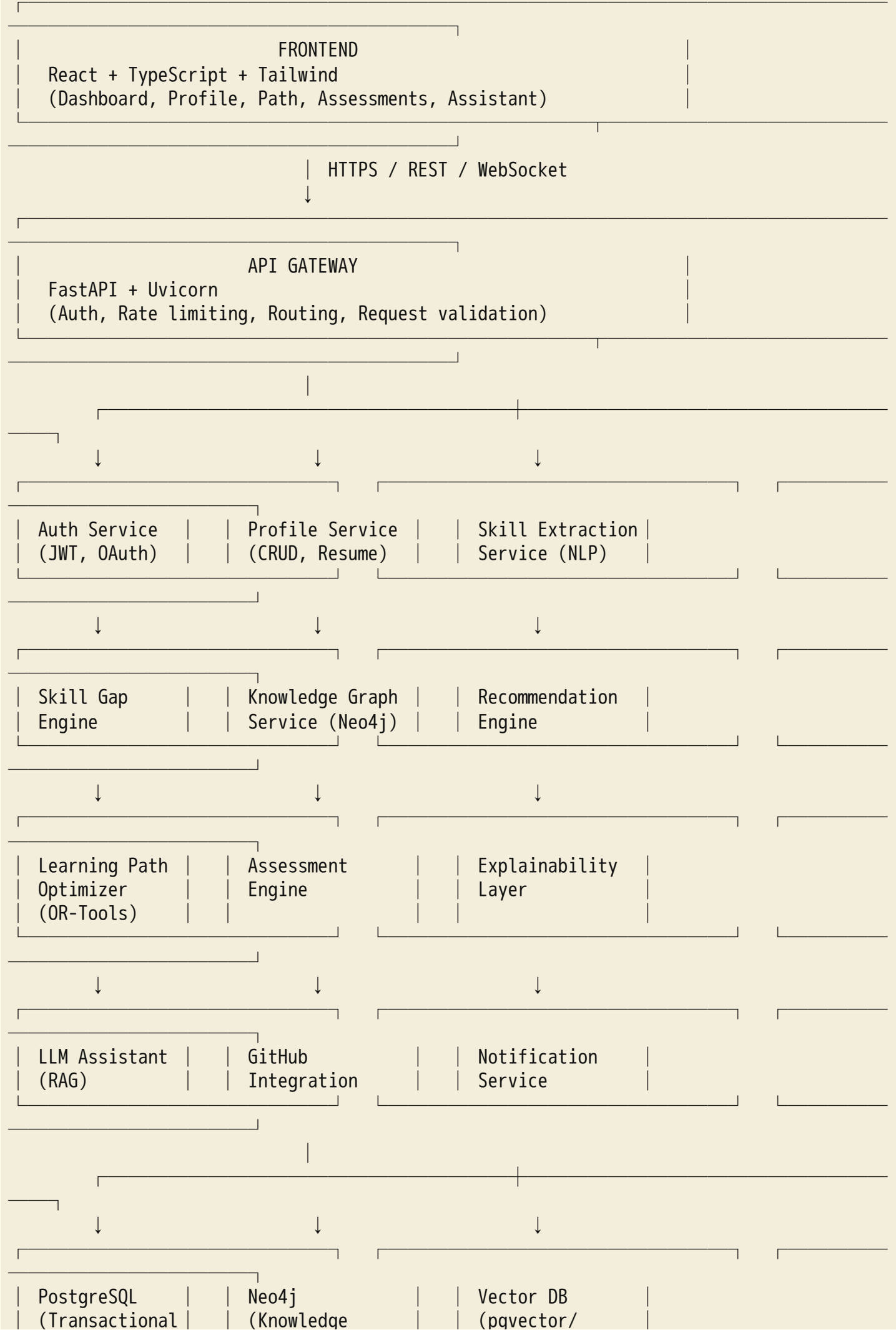
Feature	Description
Current vs. Required	Bar comparison
Gap visualization	Color-coded (red = high)
Confidence indicator	Opacity or badge
Evidence tooltip	Hover to see sources
Progress over time	Line chart
Target role overlay	Switch between roles

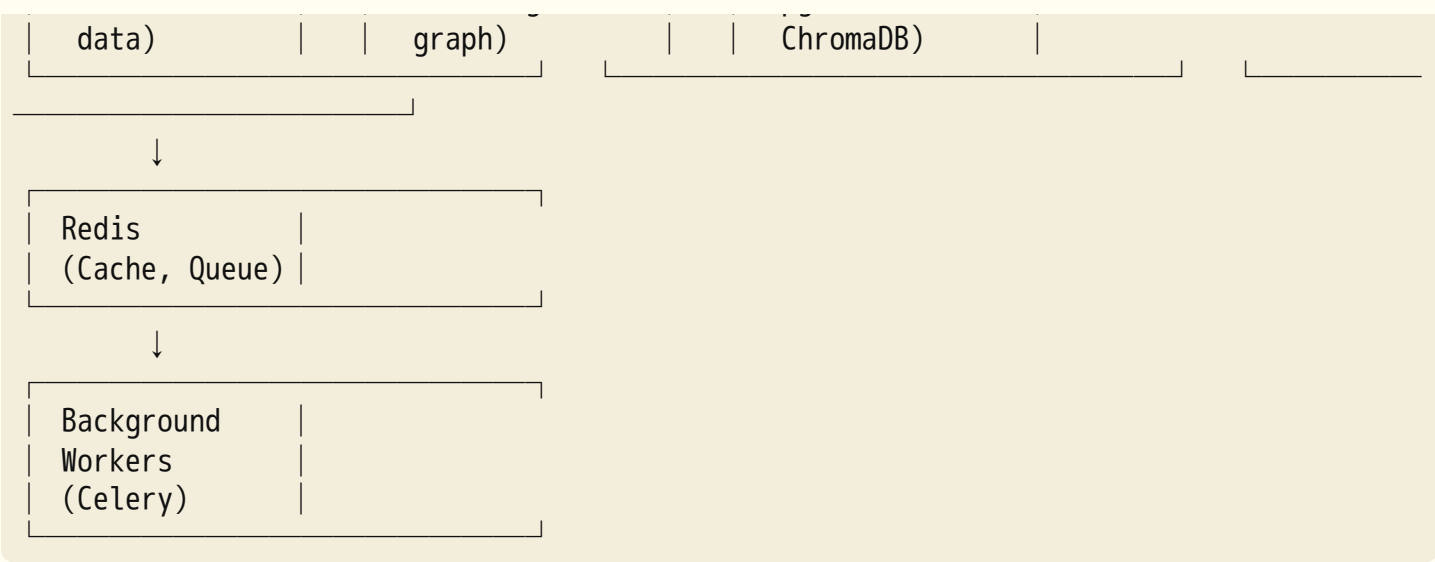
22.4 Progress Tracking



23. System Architecture

23.1 High-Level Architecture





23.2 Component Descriptions

Component	Purpose	Technology
Frontend	User interface	React, TypeScript, Tailwind
API Gateway	Request routing, auth	FastAPI
Auth Service	Authentication, authorization	JWT, OAuth2
Profile Service	Employee CRUD, resume storage	FastAPI + PostgreSQL
Skill Extraction	NLP pipeline	spaCy, Transformers
Skill Gap Engine	Gap computation	Python, NumPy
Knowledge Graph	Skill relationships	Neo4j
Recommendation Engine	Hybrid scoring	Python, Scikit-learn
Learning Path Optimizer	Sequence optimization	OR-Tools
Assessment Engine	Quiz delivery, scoring	FastAPI
Explainability Layer	Reason generation	Python
LLM Assistant	Conversational AI	LangChain + LLM API
GitHub Integration	Code analysis	GitHub API
Notification Service	Email/push	Celery + Redis
PostgreSQL	Transactional data	PostgreSQL 15
Neo4j	Graph data	Neo4j 5
Vector DB	Embeddings	pgvector or ChromaDB
Redis	Cache, message queue	Redis 7

Component	Purpose	Technology
Background Workers	Async tasks	Celery

23.3 Data Flow

1. User uploads resume → Profile Service → S3/DB
 2. Resume processing → Skill Extraction Service → NLP pipeline
 3. Skills stored → PostgreSQL + Vector DB (embeddings)
 4. User selects role → Skill Gap Engine → Gap scores
 5. Gaps → Knowledge Graph → Prerequisite expansion
 6. Resources retrieved → Recommendation Engine → Ranked list
 7. Ranked list → Learning Path Optimizer → Ordered path
 8. Path stored → PostgreSQL → Frontend
 9. User completes assessment → Assessment Engine → Updated profile
 10. Profile update → Adaptive engine → Re-optimized path

24. Database Design

24.1 PostgreSQL Schema

Core Entities

```
sql

-- Employees
CREATE TABLE employees (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  full_name VARCHAR(255) NOT NULL,
  job_title VARCHAR(255),
  department VARCHAR(255),
  years_experience DECIMAL(3,1),
  available_hours_per_week INT DEFAULT 10,
  learning_preferences JSONB,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

-- Skills
CREATE TABLE skills (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) UNIQUE NOT NULL,
  category VARCHAR(100),
  description TEXT,
  parent_skill_id UUID REFERENCES skills(id),
  created_at TIMESTAMP DEFAULT NOW()
```

```

);

-- Skill Levels (proficiency definitions)
CREATE TABLE skill_levels (
    id SERIAL PRIMARY KEY,
    level INT NOT NULL CHECK (level BETWEEN 1 AND 5),
    label VARCHAR(50) NOT NULL,
    description TEXT
);

-- Roles
CREATE TABLE roles (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    title VARCHAR(255) NOT NULL,
    category VARCHAR(100),
    description TEXT,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Role Skills (required skills for role)
CREATE TABLE role_skills (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    role_id UUID REFERENCES roles(id) ON DELETE CASCADE,
    skill_id UUID REFERENCES skills(id) ON DELETE CASCADE,
    required_level INT NOT NULL CHECK (required_level BETWEEN 1 AND 5),
    importance DECIMAL(3,2) DEFAULT 0.5 CHECK (importance BETWEEN 0 AND 1),
    criticality DECIMAL(3,2) DEFAULT 0.5 CHECK (criticality BETWEEN 0 AND 1),
    UNIQUE(role_id, skill_id)
);

-- Employee Skills (current profile)
CREATE TABLE employee_skills (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    employee_id UUID REFERENCES employees(id) ON DELETE CASCADE,
    skill_id UUID REFERENCES skills(id) ON DELETE CASCADE,
    current_level DECIMAL(3,1) NOT NULL CHECK (current_level BETWEEN 0 AND 5),
    confidence DECIMAL(3,2) DEFAULT 0.5 CHECK (confidence BETWEEN 0 AND 1),
    last_updated TIMESTAMP DEFAULT NOW(),
    UNIQUE(employee_id, skill_id)
);

-- Skill Evidence
CREATE TABLE skill_evidence (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    employee_id UUID REFERENCES employees(id) ON DELETE CASCADE,
    skill_id UUID REFERENCES skills(id) ON DELETE CASCADE,
    source_type VARCHAR(50) NOT NULL, -- RESUME, GITHUB, COURSE, ASSESSMENT, PROJECT, CERT,
    WORK, SELF
    source_id UUID,

```

```

    raw_text TEXT,
    extracted_level DECIMAL(3,1),
    reliability DECIMAL(3,2) DEFAULT 0.5,
    strength DECIMAL(3,2) DEFAULT 0.5,
    recency DECIMAL(3,2) DEFAULT 1.0,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Courses
CREATE TABLE courses (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    title VARCHAR(500) NOT NULL,
    provider VARCHAR(255),
    description TEXT,
    url VARCHAR(500),
    duration_hours DECIMAL(5,1),
    difficulty INT CHECK (difficulty BETWEEN 1 AND 5),
    rating DECIMAL(2,1),
    completion_rate DECIMAL(3,2),
    created_at TIMESTAMP DEFAULT NOW()
);

-- Course Skills
CREATE TABLE course_skills (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    course_id UUID REFERENCES courses(id) ON DELETE CASCADE,
    skill_id UUID REFERENCES skills(id) ON DELETE CASCADE,
    proficiency_gain DECIMAL(3,1),
    coverage DECIMAL(3,2) DEFAULT 1.0,
    UNIQUE(course_id, skill_id)
);

-- Projects
CREATE TABLE projects (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    title VARCHAR(500) NOT NULL,
    description TEXT,
    difficulty INT CHECK (difficulty BETWEEN 1 AND 5),
    duration_hours DECIMAL(5,1),
    portfolio_value DECIMAL(3,2),
    created_at TIMESTAMP DEFAULT NOW()
);

-- Project Skills
CREATE TABLE project_skills (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID REFERENCES projects(id) ON DELETE CASCADE,
    skill_id UUID REFERENCES skills(id) ON DELETE CASCADE,
    proficiency_gain DECIMAL(3,1),

```

```

        UNIQUE(project_id, skill_id)
    );

-- Mentors
CREATE TABLE mentors (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    employee_id UUID REFERENCES employees(id) ON DELETE CASCADE,
    bio TEXT,
    years_experience INT,
    availability_hours_per_month INT,
    max_mentees INT DEFAULT 5,
    current_mentees INT DEFAULT 0,
    rating DECIMAL(2,1),
    created_at TIMESTAMP DEFAULT NOW()
);

-- Mentor Skills
CREATE TABLE mentor_skills (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    mentor_id UUID REFERENCES mentors(id) ON DELETE CASCADE,
    skill_id UUID REFERENCES skills(id) ON DELETE CASCADE,
    expertise_level INT CHECK (expertise_level BETWEEN 1 AND 5),
    years_experience INT,
    UNIQUE(mentor_id, skill_id)
);

-- Assessments
CREATE TABLE assessments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    title VARCHAR(500) NOT NULL,
    type VARCHAR(50) NOT NULL, -- MCQ, CODING, CONCEPTUAL, PRACTICAL
    skill_id UUID REFERENCES skills(id),
    difficulty INT CHECK (difficulty BETWEEN 1 AND 5),
    time_limit_minutes INT,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Assessment Questions
CREATE TABLE assessment_questions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    assessment_id UUID REFERENCES assessments(id) ON DELETE CASCADE,
    question_text TEXT NOT NULL,
    question_type VARCHAR(50) NOT NULL,
    options JSONB,
    correct_answer TEXT,
    difficulty DECIMAL(3,2),
    weight DECIMAL(3,2) DEFAULT 1.0
);

```

```

-- Assessment Results
CREATE TABLE assessment_results (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    employee_id UUID REFERENCES employees(id) ON DELETE CASCADE,
    assessment_id UUID REFERENCES assessments(id) ON DELETE CASCADE,
    score DECIMAL(5,2),
    max_score DECIMAL(5,2),
    proficiency_estimate DECIMAL(3,1),
    answers JSONB,
    completed_at TIMESTAMP DEFAULT NOW()
);

-- Learning Paths
CREATE TABLE learning_paths (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    employee_id UUID REFERENCES employees(id) ON DELETE CASCADE,
    target_role_id UUID REFERENCES roles(id),
    status VARCHAR(50) DEFAULT 'ACTIVE', -- ACTIVE, COMPLETED, PAUSED
    total_hours DECIMAL(6,1),
    estimated_weeks INT,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Learning Path Items
CREATE TABLE learning_path_items (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    path_id UUID REFERENCES learning_paths(id) ON DELETE CASCADE,
    sequence_order INT NOT NULL,
    item_type VARCHAR(50) NOT NULL, -- COURSE, PROJECT, ASSESSMENT, MILESTONE
    item_id UUID NOT NULL,
    skill_id UUID REFERENCES skills(id),
    status VARCHAR(50) DEFAULT 'NOT_STARTED',
    started_at TIMESTAMP,
    completed_at TIMESTAMP,
    score DECIMAL(5,2)
);

-- Recommendations
CREATE TABLE recommendations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    employee_id UUID REFERENCES employees(id) ON DELETE CASCADE,
    resource_type VARCHAR(50) NOT NULL,
    resource_id UUID NOT NULL,
    score DECIMAL(5,4),
    score_breakdown JSONB,
    reason TEXT,
    created_at TIMESTAMP DEFAULT NOW(),
    expires_at TIMESTAMP

```

```

);

-- Feedback
CREATE TABLE feedback (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  employee_id UUID REFERENCES employees(id) ON DELETE CASCADE,
  recommendation_id UUID REFERENCES recommendations(id),
  rating INT CHECK (rating BETWEEN 1 AND 5),
  comment TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);

-- Course Completions
CREATE TABLE course_completions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  employee_id UUID REFERENCES employees(id) ON DELETE CASCADE,
  course_id UUID REFERENCES courses(id) ON DELETE CASCADE,
  completed_at TIMESTAMP DEFAULT NOW(),
  grade DECIMAL(5,2),
  certificate_url VARCHAR(500)
);

-- Skill Prerequisites (also in Neo4j)
CREATE TABLE skill_prerequisites (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  skill_id UUID REFERENCES skills(id) ON DELETE CASCADE,
  prerequisite_skill_id UUID REFERENCES skills(id) ON DELETE CASCADE,
  strength DECIMAL(3,2) DEFAULT 1.0 CHECK (strength BETWEEN 0 AND 1),
  UNIQUE(skill_id, prerequisite_skill_id)
);

```

24.2 Neo4j Graph Schema

cypher

```

// Nodes
(:Employee {id, name, email})
(:Skill {id, name, category})
(:Course {id, title, provider})
(:Project {id, title, difficulty})
(:Mentor {id, name})
(:TargetRole {id, title})
(:Assessment {id, type})

// Relationships
(:TargetRole)-[:REQUIRES {importance, required_level}]->(:Skill)
(:Skill)-[:PREREQUISITE_OF {strength}]->(:Skill)
(:Skill)-[:RELATED_TO {similarity}]->(:Skill)

```

```
(:Course)-[:TEACHES {proficiency_gain}]->(:Skill)
(:Project)-[:PRACTICES {proficiency_gain}]->(:Skill)
(:Mentor)-[:EXPERT_IN {level, years}]->(:Skill)
(:Course)-[:RECOMMENDED_FOR]->(:TargetRole)
(:Employee)-[:HAS_SKILL {level, confidence}]->(:Skill)
(:Employee)-[:COMPLETED]->(:Course)
(:Mentor)-[:MENTORS]->(:Employee)
```

24.3 Vector Database

Purpose: Store embeddings for semantic similarity.

Collection	Content	Dimension
skill_embeddings	Skill descriptions	384
course_embeddings	Course descriptions	384
project_embeddings	Project descriptions	384
resume_embeddings	Resume chunks	384
jd_embeddings	Job description chunks	384

Technology: pgvector (if using PostgreSQL) or ChromaDB.

24.4 Data Distribution

Data	Storage	Reason
Employee profiles	PostgreSQL	Transactional
Skills taxonomy	PostgreSQL	Relational
Role requirements	PostgreSQL	Relational
Prerequisites	Neo4j + PostgreSQL	Graph traversal
Course-skill mapping	Neo4j + PostgreSQL	Both
Embeddings	Vector DB	Similarity search
Learning paths	PostgreSQL	Transactional
Assessments	PostgreSQL	Transactional
Recommendations	PostgreSQL	Audit trail
Graph relationships	Neo4j	Traversal performance

25. API Design

25.1 Authentication

Method	Endpoint	Request	Response	Purpose
POST	/api/auth/register	{email, password, full_name, role}	{user_id, token}	Register
POST	/api/auth/login	{email, password}	{token, user}	Login
POST	/api/auth/refresh	{refresh_token}	{token}	Refresh token
POST	/api/auth/logout	—	{success}	Logout

25.2 Employee Profiles

Method	Endpoint	Request	Response	Purpose
GET	/api/employees/me	—	{profile}	Get own profile
PUT	/api/employees/me	{profile_data}	{profile}	Update profile
POST	/api/employees/me/resume	multipart/form-data	{extracted_skills}	Upload resume
GET	/api/employees/me/skills	—	[{skill, level, confidence}]	Get skills
POST	/api/employees/me/skills	{skill_id, level}	{skill}	Add skill
PUT	/api/employees/me/skills/{id}	{level}	{skill}	Update skill
DELETE	/api/employees/me/skills/{id}	—	{success}	Remove skill

25.3 Resume Processing

Method	Endpoint	Request	Response	Purpose
POST	/api/resume/extract	{file}	{skills, education, experience}	Extract from resume
POST	/api/resume/parse-jd	{text}	{required_skills, preferred_skills}	Parse JD

25.4 Skill Gap Analysis

Method	Endpoint	Request	Response	Purpose
POST	/api/gap/analyze	{target_role_id}	{gaps: [{skill, gap, priority}]}	Analyze gaps
GET	/api/gap/current	—	{gaps}	Get current gaps

25.5 Roles

Method	Endpoint	Request	Response	Purpose
GET	/api/roles	—	[[role]]	List roles
GET	/api/roles/{id}	—	{role, skills}	Role details
GET	/api/roles/{id}/skills	—	[[skill, required_level]]	Role skills

25.6 Courses

Method	Endpoint	Request	Response	Purpose
GET	/api/courses	?skill_id=&difficulty=	[[course]]	List courses
GET	/api/courses/{id}	—	{course, skills}	Course details
POST	/api/courses/{id}/enroll	—	{enrollment}	Enroll

25.7 Projects

Method	Endpoint	Request	Response	Purpose
GET	/api/projects	?skill_id=&difficulty =	{{project}}	List projects
GET	/api/projects/{id}	—	{project, skills}	Project details
POST	/api/projects/{id}/start	—	{progress}	Start project

25.8 Mentors

Method	Endpoint	Request	Response	Purpose
GET	/api/mentors	?skill_id =	{{mentor}}	List mentors
GET	/api/mentors/{id}	—	{mentor, skills}	Mentor details
POST	/api/mentors/{id}/request	{message}	{request}	Request mentorship
GET	/api/mentors/recommended	—	{{mentor, score, reason}}	Recommended mentors

25.9 Recommendations

Method	Endpoint	Request	Response	Purpose
GET	/api/recommendations/courses	?limit=10	{{course, score, reason}}	Course recs
GET	/api/recommendations/projects	?limit=10	{{project, score, reason}}	Project recs
GET	/api/recommendations/mentors	?limit=5	{{mentor, score, reason}}	Mentor recs
GET	/api/recommendations/all	—	{courses, projects, mentors}	All recs

25.10 Learning Paths

Method	Endpoint	Request	Response	Purpose
POST	/api/paths/generate	{target_role_id, constraints}	{path}	Generate path
GET	/api/paths/current	—	{path, items}	Get current path
GET	/api/paths/{id}	—	{path, items}	Path details
PUT	/api/paths/{id}/items/{item_id}	{status}	{item}	Update item
POST	/api/paths/{id}/reoptimize	—	{path}	Re-optimize

25.11 Assessments

Method	Endpoint	Request	Response	Purpose
GET	/api/assessments	?skill_id =	[[assessment]]	List assessments
GET	/api/assessments/{id}	—	{assessment, questions}	Get assessment
POST	/api/assessments/{id}/submit	{answers}	{score, feedback}	Submit
GET	/api/assessments/results	—	[[result]]	Get results

25.12 Progress

Method	Endpoint	Request	Response	Purpose
GET	/api/progress/summary	—	{completed, in_progress, skills_gained}	Summary
GET	/api/progress/skills	—	[[skill, level, progress]]	Skill progress
GET	/api/progress/timeline	—	[[date, event]]	Timeline

25.13 GitHub Integration

Method	Endpoint	Request	Response	Purpose
POST	/api/github/connect	{oauth_token}	{connected}	Connect
GET	/api/github/repos	—	[[repo]]	List repos
POST	/api/github/analyze	{repo_ids}	{skills}	Analyze repos
GET	/api/github/skills	—	[[skill, confidence]]	Extracted skills

25.14 AI Assistant

Metho d	Endpoint	Request	Response	Purpose
POST	/api/assistant/chat	{message, contex t}	{response, source s}	Chat
GET	/api/assistant/histor y	—	[[message]]	History
POST	/api/assistant/explai n	{resource_id, type}	{explanation}	Explain re c

25.15 What-If Simulation

Metho d	Endpoint	Request	Response	Purpose
POST	/api/simulation/comp are	{role_id s}	[[role, coverage, gaps, eff ort]]	Compare rol es
GET	/api/simulation/roles	—	[[role]]	Available rol es

26. Frontend Design

26.1 Employee Dashboard

Components:

- Welcome header with profile summary
- Skill Digital Twin (compact)
- Current learning path progress

- Recommended next actions
- Recent assessments
- Upcoming mentor sessions

Interactions:

- Click skill → detail modal
- Click path item → resource page
- Click recommendation → detail

26.2 Skill Profile

Components:

- Skill list with proficiency bars
- Confidence indicators
- Evidence sources (expandable)
- Add/edit skill form
- Resume upload area

Interactions:

- Hover skill → evidence tooltip
- Click skill → full evidence view
- Drag to reorder priority

26.3 Skill Gap Visualization

Components:

- Gap heatmap (skill vs. required)
- Priority ranking list
- Filter by category
- Target role selector

Interactions:

- Click gap → recommended resources
- Toggle between roles
- Export gap report

26.4 Career/Role Explorer

Components:

- Role cards with coverage %
- Search/filter roles
- Role detail view
- "Set as target" button

Interactions:

- Click role → detail
- Compare roles
- Set target → triggers gap analysis

26.5 Recommended Learning Path

Components:

- Visual path (timeline/flowchart)
- Item cards (course/project/assessment)
- Progress indicators
- Milestone markers
- Time estimates

Interactions:

- Click item → detail
- Mark complete
- Reorder (if allowed)
- Re-optimize button

26.6 Course Details

Components:

- Course info (provider, duration, difficulty)
- Skills taught
- Prerequisites
- Match score + explanation
- Enroll button

26.7 Project Recommendations

Components:

- Project cards

- Skills practiced
- Difficulty
- Duration
- Portfolio value
- Start button

26.8 Mentor Recommendations

Components:

- Mentor cards
- Expertise overlap
- Match score
- Availability
- Request button
- Explanation

26.9 Assessment Dashboard

Components:

- Available assessments
- Completed assessments
- Scores + feedback
- Skill impact
- Retake option

26.10 Progress Tracking

Components:

- Skill progress charts
- Path completion %
- Time invested
- Skills gained
- Milestones achieved

26.11 Skill Digital Twin

Components:

- Full skill visualization

- Current vs. required
- Gap highlighting
- Confidence indicators
- Evidence drill-down
- Time-series progress

26.12 AI Learning Assistant

Components:

- Chat interface
- Context sidebar (current path, gaps)
- Suggested questions
- Source citations
- Action buttons (e.g., "Start this course")

26.13 What-If Career Simulation

Components:

- Role comparison table
- Side-by-side skill gaps
- Effort estimates
- Path previews
- Export comparison

27. Recommendation Explainability UI

27.1 Design Principles

1. **Show, don't tell** — Visual score breakdown
2. **Evidence first** — Link to data
3. **Confidence-aware** — Show uncertainty
4. **Actionable** — "Take this next"

27.2 Recommendation Card

Field	Type	Example
email	String	"alex@example.com"
job_title	String	"Software Engineer"
years_experience	Float	2.0
available_hours	Int	10

Data Generation: Synthetic profiles with realistic skill distributions.

28.2 Skills

Field	Type	Example
id	UUID	s1...
name	String	"Python"
category	String	"Programming"
description	Text	"General-purpose programming language"

Sources: ESCO, O*NET, LinkedIn Skills, Stack Overflow tags.

28.3 Roles

Field	Type	Example
id	UUID	r1...
title	String	"ML Engineer"
category	String	"AI/ML"
required_skills	JSONB	[{skill, level, importance}]

Sources: O*NET, LinkedIn job postings, Kaggle job datasets.

28.4 Courses

Field	Type	Example
id	UUID	c1...
title	String	"Deep Learning Specialization"
provider	String	"Coursera"
duration_hours	Float	120

Field	Type	Example
difficulty	Int	4
skills	JSONB	[{skill, gain}]

Sources: Coursera, Udemy, edX APIs (where available); manual curation.

28.5 Projects

Field	Type	Example
id	UUID	p1...
title	String	"Build a RAG Chatbot"
difficulty	Int	3
duration_hours	Float	20
skills	JSONB	[{skill, gain}]

Sources: Manual curation from GitHub, Kaggle, personal projects.

28.6 Mentors

Field	Type	Example
id	UUID	m1...
name	String	"Dr. Sarah Chen"
expertise	JSONB	[{skill, level, years}]
availability	Int	4

Sources: Synthetic (for demo) or opt-in real mentors.

28.7 Assessments

Field	Type	Example
id	UUID	a1...
skill	String	"Python"
questions	JSONB	[{text, type, answer}]

Sources: Manual creation, Open edX question banks.

28.8 Skill Prerequisites

Field	Type	Example
skill	String	"RAG"
prerequisite	String	"LLMs"
strength	Float	0.9

Sources: Expert curation, curriculum analysis, LLM-assisted extraction.

28.9 Historical Interactions

Field	Type	Example
employee_id	UUID	e1...
resource_id	UUID	c1...
action	String	"completed"
rating	Int	5
timestamp	DateTime	2024-01-15

Generation: Simulated user interactions based on skill profiles.

28.10 Data Generation Strategy

- 1. **Seed with real data** where available (ESCO, O*NET, public course catalogs).
- 2. **Synthesize employees** with realistic skill distributions.
- 3. **Generate interactions** using a simulated user model.
- 4. **Validate** with domain experts.
- 5. **Augment** with LLM-generated variations.

28.11 Public Datasets

Dataset	Use
ESCO	Skill taxonomy
O*NET	Role-skill mapping
Kaggle Job Postings	JD parsing
Coursera Catalog	Course data

Dataset	Use
GitHub Archive	Project evidence
Stack Overflow Survey	Skill trends

29. Machine Learning Methodology

29.1 Component Responsibilities

Component	Approach	Why
Skill extraction	NER + Dictionary + Embeddings	High precision
Skill normalization	Rules + Embeddings	Handle synonyms
Proficiency estimation	Rules + Regression	Interpretable
Skill gap	Rule-based formula	Transparent
Evidence aggregation	Weighted average	Explainable
Semantic similarity	Sentence Transformers	SOTA
Collaborative filtering	ALS / Implicit	Scalable
Hybrid ranking	Weighted sum / Learn-to-rank	Flexible
Prerequisite ordering	Graph algorithms	Deterministic
Path optimization	OR-Tools CP-SAT	Optimal
Explanation	Rules + LLM	Grounded
Assistant	RAG + LLM	Contextual

29.2 Rule-Based Logic

- Skill normalization rules
- Proficiency estimation from context
- Gap formula
- Prerequisite ordering
- Difficulty fit
- Explanation generation

29.3 Machine Learning

- NER for skill extraction
- Sentence transformers for embeddings
- Collaborative filtering for CF score
- Learn-to-rank for weight tuning
- Clustering for user segmentation

29.4 Knowledge Graph Reasoning

- Prerequisite traversal
- Path finding
- Similarity inference
- Resource-skill mapping

29.5 Optimization

- Constraint programming (OR-Tools)
- Graph algorithms (topological sort)
- Linear programming (time allocation)

29.6 LLM Functionality

- Skill extraction assistance (edge cases)
- Natural-language explanations
- Conversational assistant
- Summarization
- Question generation

29.7 Distinction Summary

Layer	Deterministic	ML	KG	Optimization	LLM
Skill extraction	✓	✓	—	—	✓ (assist)
Normalization	✓	✓	—	—	—
Proficiency	✓	✓	—	—	—
Gap	✓	—	—	—	—
Evidence	✓	—	—	—	—
Similarity	—	✓	—	—	—

Layer	Deterministic	ML	KG	Optimization	LLM
CF	—	✓	—	—	—
Ranking	✓	✓	—	—	—
Prereq	—	—	✓	—	—
Ordering	—	—	✓	✓	—
Optimization	—	—	—	✓	—
Explanation	✓	—	—	—	✓
Assistant	—	—	—	—	✓

30. Evaluation Framework

30.1 Recommendation Metrics

Metric	Formula	Target
Precision@K	$\frac{\text{relevant}}{\text{top-K}}$	≥ 0.80
Recall@K	$\frac{\text{relevant}}{\text{top-K}}$	≥ 0.80
NDCG@K	$\frac{DCG@K}{IDCG@K}$	≥ 0.80
MAP@K	$\frac{1}{U}$	≥ 0.80
Diversity	$1 - \text{avg pairwise similarity}$	≥ 0.50
Coverage	$\frac{\text{recommended items}}{\text{total items}}$	≥ 0.80

30.2 Skill-Gap Metrics

Metric	Target
Skill extraction precision	≥ 0.85
Skill extraction recall	≥ 0.80
Skill extraction F1	≥ 0.82
Skill classification accuracy	≥ 0.85
Proficiency estimation MAE	≤ 0.50
Gap ranking NDCG@10	≥ 0.80

30.3 Learning-Path Metrics

Metric	Target
Skill coverage	≥ 0.90
Prerequisite violations	0
Learning time	Minimized
Path efficiency	≥ 0.85

30.4 User Metrics

Metric	Target
User satisfaction (SUS)	≥ 75
Recommendation usefulness	$\geq 4.0/5$
Explainability satisfaction	$\geq 80\%$
Path completion rate	$\geq 60\%$

30.5 Experimental Design

Compare 5 approaches:

#	Approach	Description
1	Popularity-based	Most popular courses
2	Content-based	Skill similarity
3	Semantic	Embedding similarity
4	Knowledge-graph	Prerequisite + graph
5	Hybrid (proposed)	All signals

Evaluation:

- Offline: historical interaction data
- Online: A/B test with users
- Ablation: remove one component at a time

30.6 Ablation Study

Configuration	Precision@10	NDCG@10
Full hybrid	0.72	0.82
– Semantic	0.68	0.78
– Gap	0.65	0.75
– Prereq	0.70	0.80
– CF	0.71	0.81
– KG	0.70	0.80

31. Research Contribution

31.1 Genuine Research Contributions

#	Contribution	Why Novel
1	Evidence-Weighted Skill Confidence Model	Combines multi-source evidence with reliability, strength, recency into a single confidence score; novel formulation
2	Prerequisite-Aware Learning Path Optimization	Formulates path generation as constrained optimization with prerequisite DAG constraints; novel application
3	Hybrid Recommendation Framework for Competency Development	7-signal hybrid combining semantic, gap, prerequisite, difficulty, preference, CF, KG; novel in EdTech
4	Explainable Recommendation Taxonomy for Learning	Deterministic explanation layer grounded in evidence; addresses LLM hallucination
5	Adaptive Learning Path Re-optimization	Dynamic re-optimization based on assessment feedback; continuous adaptation
6	Skill Digital Twin	Dynamic, evidence-backed visualization of competence; novel UI concept

31.2 Ordinary Software Features (Not Research)

- User authentication
- Profile CRUD
- Resume upload
- Course catalog

- Assessment delivery
- Dashboard UI
- Notification system

31.3 Publication Targets

- **EdTech conferences:** LAK, EDM, AIED
- **Recommender systems:** RecSys, ACM TOIS
- **Applied ML:** NeurIPS workshops, AAAI
- **HR Tech:** HR Tech Conference, IJCAI workshops

32. Security and Privacy

32.1 Employee Data

Data	Sensitivity	Protection
Resume	High	Encrypted at rest, access-controlled
Skill profile	Medium	Encrypted, user-owned
Assessment results	Medium	Encrypted
GitHub data	Medium	OAuth-scoped
Learning history	Low	Standard

32.2 Resume Privacy

- Encrypted at rest (AES-256)
- Encrypted in transit (TLS 1.3)
- User can delete anytime
- Not shared without consent
- Processed in isolated environment

32.3 GitHub Permissions

- OAuth with minimal scopes (`read:user` , `public_repo`)
- User can revoke anytime
- No write access
- Tokens encrypted

- No code stored permanently

32.4 Authentication

- JWT with short expiry (15 min)
- Refresh tokens (7 days)
- Password hashing (bcrypt, cost 12)
- Optional MFA
- OAuth2 for SSO

32.5 Authorization

- RBAC: Employee, Manager, Mentor, HR, Admin
- Row-level security in PostgreSQL
- API-level permission checks
- Resource ownership validation

32.6 Encryption

Layer	Method
Transit	TLS 1.3
At rest	AES-256
Passwords	bcrypt
Tokens	JWT (RS256)
Database	Transparent data encryption

32.7 Data Retention

Data	Retention
Profile	Until deletion
Resume	Until deletion
Assessments	2 years
Logs	90 days
Analytics	Anonymized, indefinite

32.8 Access Control

- Least privilege principle
- Audit logging
- Session management
- IP allowlisting (admin)
- Rate limiting

32.9 Sensitive HR Information

- Manager sees aggregated team data only
- Individual data requires employee consent
- HR sees anonymized analytics by default
- Admin access logged and audited

32.10 LLM Data Privacy

- No PII sent to LLM APIs
- Anonymize before sending
- Use local models where possible
- Opt-out option
- Clear data processing agreement

33. Responsible AI

33.1 Recommendation Bias

Risk	Mitigation
Popularity bias	Diversity constraints in ranking
Skill bias	Balanced taxonomy; audit recommendations
Demographic bias	No demographic features in model
Role bias	Multiple role frameworks

33.2 Skill Inference Uncertainty

- Always show confidence scores
- Flag low-confidence inferences
- Prompt for assessment when uncertain

- Never present inferred skills as facts

33.3 Incomplete Evidence

- Show evidence gaps
- Suggest ways to add evidence
- Weight confidence accordingly
- Don't over-recommend based on thin evidence

33.4 Over-Reliance on Self-Reported Data

- Self-reports have lowest reliability (0.3)
- Encourage evidence addition
- Flag self-reported-only skills
- Suggest assessments

33.5 LLM Hallucinations

- Ground LLM in retrieved context
- Require citations
- Deterministic layer for decisions
- LLM only for natural language
- Human review for high-stakes

33.6 Explainability

- Every recommendation has reason
- Reasons are evidence-linked
- Confidence communicated
- Alternatives shown
- No black-box decisions

33.7 Human Oversight

- Manager can override recommendations
- HR can adjust taxonomy
- Admin can audit decisions
- User can provide feedback
- System suggests, human decides

33.8 Uncertainty Communication

Confidence	Display
0.9–1.0	"High confidence"
0.7–0.9	"Moderate confidence"
0.5–0.7	"Low confidence – assessment recommended"
< 0.5	"Very low confidence – please verify"

34. Non-Functional Requirements

34.1 Performance

Requirement	Target
API response time (p95)	< 500ms
Skill extraction	< 10s per resume
Gap analysis	< 2s
Path generation	< 3s
Recommendation	< 1s
Page load	< 2s

34.2 Scalability

Requirement	Target
Concurrent users	1,000
Employees	100,000
Skills	10,000
Courses	50,000
Recommendations/day	1,000,000

34.3 Availability

Requirement	Target
Uptime	99.5%

Requirement	Target
Recovery time	< 1 hour
Backup frequency	Daily
Disaster recovery	Cross-region

34.4 Security

- See Section 32

34.5 Maintainability

Requirement	Target
Code coverage	≥ 80%
Documentation	Complete API docs
Modularity	Microservices
CI/CD	Automated
Logging	Structured

34.6 Explainability

- Every recommendation explainable
- Score breakdown available
- Evidence links
- Confidence shown

34.7 Observability

Tool	Purpose
Prometheus	Metrics
Grafana	Dashboards
ELK Stack	Logs
Sentry	Errors
OpenTelemetry	Tracing

34.8 Usability

Requirement	Target
SUS score	≥ 75
Onboarding time	< 5 min
Mobile responsive	Yes
Accessibility	WCAG 2.1 AA

35. Technology Stack

35.1 Frontend

Technology	Why
React 18	Component-based, mature ecosystem
TypeScript	Type safety, maintainability
Tailwind CSS	Rapid UI development
Recharts / D3	Data visualization
React Query	Server state management
Zustand	Client state management

35.2 Backend

Technology	Why
FastAPI	Async, fast, OpenAPI docs
Python 3.11	ML ecosystem
Pydantic	Data validation
SQLAlchemy	ORM
Alembic	Migrations
Celery	Background tasks
Redis	Cache, queue

35.3 Databases

Technology	Why
PostgreSQL 15	ACID, JSONB, reliable
Neo4j 5	Native graph, Cypher
pgvector	Vector search in PostgreSQL
Redis 7	Cache, sessions, queue

35.4 ML/NLP

Technology	Why
spaCy	NER, fast
Sentence Transformers	Embeddings
Scikit-learn	Classical ML
PyTorch	Deep learning (if needed)
Hugging Face	Model hub

35.5 Optimization

Technology	Why
OR-Tools	Constraint programming
NetworkX	Graph algorithms

35.6 LLM

Technology	Why
LangChain	RAG orchestration
OpenAI / Gemini / Groq	LLM APIs
ChromaDB / pgvector	Vector store

35.7 DevOps

Technology	Why
Docker	Containerization
Docker Compose	Local orchestration

Technology	Why
GitHub Actions	CI/CD
Prometheus + Grafana	Monitoring
Sentry	Error tracking

35.8 Development

Technology	Why
Git	Version control
pytest	Testing
Black + Ruff	Formatting, linting
mypy	Type checking
pre-commit	Git hooks

36. Project Folder Structure

```

skillpath-ai/
├── frontend/                                     # React + TypeScript
│   ├── public/
│   └── src/
│       ├── components/                         # Reusable UI components
│       │   ├── common/
│       │   ├── dashboard/
│       │   ├── profile/
│       │   ├── gap/
│       │   ├── path/
│       │   ├── recommendations/
│       │   └── assistant/
│       ├── pages/                             # Route pages
│       ├── hooks/                             # Custom hooks
│       ├── services/                         # API clients
│       ├── store/                           # State management
│       ├── types/                           # TypeScript types
│       ├── utils/                           # Utilities
│       └── styles/                           # Global styles
│   ├── package.json
│   └── tsconfig.json
├── backend/                                    # FastAPI
│   ├── app/
│   │   ├── api/                              # API routes
│   │   │   ├── v1/
│   │   │   │   ├── auth.py
│   │   │   │   ├── employees.py
│   │   │   │   ├── resume.py
│   │   │   │   ├── skills.py
│   │   │   │   ├── gap.py
│   │   │   │   ├── roles.py
│   │   │   │   ├── courses.py
│   │   │   │   ├── projects.py
│   │   │   │   ├── mentors.py
│   │   │   │   ├── recommendations.py
│   │   │   │   ├── paths.py
│   │   │   │   ├── assessments.py
│   │   │   │   ├── progress.py
│   │   │   │   ├── github.py
│   │   │   │   ├── assistant.py
│   │   │   │   └── simulation.py
│   │   │   └── deps.py                      # Dependencies
│   │   ├── core/                            # Config, security
│   │   │   ├── config.py
│   │   │   ├── security.py
│   │   │   └── logging.py
│   │   ├── models/                          # SQLAlchemy models
│   │   ├── schemas/                        # Pydantic schemas
│   │   ├── services/                       # Business logic
│   │   │   ├── profile_service.py
│   │   │   ├── skill_service.py
│   │   │   ├── gap_service.py
│   │   │   ├── recommendation_service.py
│   │   │   ├── path_service.py
│   │   │   └── assessment_service.py

```

└── evidence_service.py	
└── db/	# Database
└── session.py	
└── base.py	
└── main.py	# App entry
└── alembic/	# Migrations
└── tests/	
└── requirements.txt	
└── Dockerfile	
ml/	# Machine Learning
└── skill_extraction/	# NLP pipeline
└── extractor.py	
└── normalizer.py	
└── taxonomy_mapper.py	
└── proficiency_estimator.py	
└── embeddings/	# Sentence transformers
└── encoder.py	
└── similarity.py	
└── evidence/	# Evidence aggregation
└── aggregator.py	
└── confidence.py	
└── models/	# Trained models
└── notebooks/	# Experiments
└── requirements.txt	
recommendation/	# Recommendation engine
└── content_based.py	
└── semantic.py	
└── collaborative.py	
└── knowledge_graph.py	
└── gap_relevance.py	
└── difficulty_fit.py	
└── hybrid.py	# Main engine
└── ranking.py	
knowledge_graph/	# Neo4j
└── client.py	
└── queries.py	
└── builder.py	# Graph construction
└── traversal.py	# Path finding
└── schema.cypher	
optimization/	# Learning path optimizer
└── path_generator.py	
└── constraint_model.py	
└── topological_sort.py	
└── reoptimizer.py	
llm/	# LLM integration
└── assistant.py	# RAG assistant
└── explainer.py	# Explanation generation
└── rag.py	# RAG pipeline
└── prompts/	# Prompt templates
database/	# Database artifacts

```

├── postgres/
│   ├── schema.sql
│   └── seed.sql
├── neo4j/
│   └── seed.cypher
├── vector/
│   └── seed_embeddings.py
├── datasets/                                # Data
│   ├── raw/
│   ├── processed/
│   ├── synthetic/
│   └── README.md
├── tests/                                  # Test suites
│   ├── unit/
│   ├── integration/
│   ├── e2e/
│   └── performance/
├── docs/                                  # Documentation
│   ├── api/
│   ├── architecture/
│   ├── research/
│   └── user_guides/
├── scripts/                               # Utility scripts
│   ├── seed_data.py
│   ├── train_models.py
│   └── evaluate.py
├── docker-compose.yml
├── .env.example
├── .gitignore
├── README.md
└── LICENSE

```

Directory Responsibilities

Directory	Responsibility
frontend/	React UI, user interactions
backend/	FastAPI, business logic, API
ml/	NLP, embeddings, evidence aggregation
recommendation/	Hybrid recommendation engine
knowledge_graph/	Neo4j client, graph algorithms
optimization/	OR-Tools path optimization
llm/	LLM integration, RAG, explanations

Directory	Responsibility
database/	Schemas, migrations, seeds
datasets/	Raw and processed data
tests/	All test types
docs/	Documentation
scripts/	Utility scripts

37. Development Roadmap

Phase 1: Foundation (Weeks 1–2)

Features:

- Project setup (frontend + backend)
- Database schema
- Authentication
- Basic employee CRUD

Dependencies: None

Output: Working auth + profile API

Testing: Unit tests for auth, profile

Phase 2: Employee Profile & Skill Extraction (Weeks 3–4)

Features:

- Resume upload
- NLP pipeline (extraction, normalization)
- Skill taxonomy
- Profile UI

Dependencies: Phase 1

Output: Resume → skills pipeline

Testing: Extraction F1 ≥ 0.80

Phase 3: Skill-Gap Engine (Weeks 5–6)

Features:

- Role definitions
- Gap computation
- Gap UI
- Evidence model

Dependencies: Phase 2

Output: Gap analysis API + UI

Testing: Gap ranking NDCG ≥ 0.75

Phase 4: Recommendation Engine (Weeks 7–8)

Features:

- Course/project/mentor data
- Content-based + semantic
- Hybrid scoring
- Recommendation UI

Dependencies: Phase 3

Output: Ranked recommendations

Testing: Precision@10 ≥ 0.65

Phase 5: Knowledge Graph (Weeks 9–10)

Features:

- Neo4j setup
- Graph schema

- Prerequisite data
- Graph queries

Dependencies: Phase 4

Output: Graph-backed recommendations

Testing: Graph query correctness

Phase 6: Learning Path Optimization (Weeks 11–12)

Features:

- Topological sort
- OR-Tools model
- Path generation
- Path UI

Dependencies: Phase 5

Output: Optimized learning paths

Testing: 0 prerequisite violations

Phase 7: Mentor/Project Recommendation (Weeks 13–14)

Features:

- Mentor matching
- Project matching
- Explainable recommendations

Dependencies: Phase 6

Output: Mentor + project recs

Testing: Match quality

Phase 8: Assessment & Adaptive Learning (Weeks 15–16)

Features:

- Assessment engine
- Adaptive path updates
- Progress tracking

Dependencies: Phase 7

Output: Adaptive system

Testing: Path updates correctly

Phase 9: Explainability & AI Assistant (Weeks 17–18)

Features:

- Explanation layer
- RAG assistant
- "Why this?" UI

Dependencies: Phase 8

Output: Explainable + conversational

Testing: Explanation accuracy

Phase 10: Evaluation & Deployment (Weeks 19–20)

Features:

- Evaluation framework
- Ablation study
- Docker deployment
- Documentation

Dependencies: Phase 9

Output: Evaluated, deployed system

Testing: Full test suite

38. MVP vs Advanced Version

38.1 MVP (Minimum Viable Product)

Goal: Demonstrate the core concept.

Feature	Included
Employee profile	✓
Resume skill extraction	✓ (basic NLP)
Skill taxonomy	✓
Role definitions	✓
Skill gap analysis	✓ (basic formula)

Feature	Included
Course recommendation	✓ (content-based)
Learning path generation	✓ (topological sort)
Basic UI	✓
Authentication	✓

Not in MVP:

- Evidence-based profiling
- Knowledge graph
- Hybrid recommendation
- Optimization
- Projects/mentors
- Assessments
- Explainability
- AI assistant
- GitHub integration
- What-If simulation
- Digital twin

38.2 Advanced Version (Research-Worthy)

Goal: Full system with research contributions.

Feature	Included
Everything in MVP	✓
Evidence-based skill profiling	✓
Multi-source evidence aggregation	✓
Knowledge graph (Neo4j)	✓
Prerequisite reasoning	✓
Hybrid recommendation (7 signals)	✓
Learning path optimization (OR-Tools)	✓
Project recommendation	✓

Feature	Included
Mentor recommendation	✓
Assessment engine	✓
Adaptive learning	✓
Explainability layer	✓
AI learning assistant (RAG)	✓
GitHub integration	✓
What-If career simulation	✓
Skill digital twin	✓
Evaluation framework	✓
Ablation study	✓

38.3 Future Work

Feature	Rationale
Real-time collaboration	Social learning
Mobile app	Accessibility
VR/AR learning	Immersive
Blockchain credentials	Verification
Federated learning	Privacy
Multi-language support	Global
Advanced IRT	Better assessments
Reinforcement learning	Adaptive sequencing
Emotion detection	Engagement
Team-level optimization	Org development

39. Example End-to-End Scenario

39.1 Employee Profile

Name: Alex Johnson
Role: Software Engineer
Experience: 2 years
Current Skills:
- Python (level 4, confidence 0.9)
- SQL (level 3.5, confidence 0.85)
- Git (level 4, confidence 0.95)
- Basic ML (level 2, confidence 0.6)
Target Role: ML Engineer
Available: 10 hours/week

39.2 Step 1: Skill Extraction

Resume processed:

- Extracted: Python, SQL, Git, Machine Learning, Docker, REST APIs
- Normalized: "ML" → "Machine Learning"
- Proficiency estimated from context:
 - Python: 4.0 (3 years experience)
 - SQL: 3.5 (2 years)
 - Git: 4.0 (daily use)
 - ML: 2.0 (course project)
- Confidence: 0.85 average

39.3 Step 2: Evidence Aggregation

Skill	Resume	GitHub	Assessment	Aggregated	Confidence
Python	4.0	4.5	—	4.27	0.93
SQL	3.5	3.0	—	3.3	0.80
Git	4.0	4.5	—	4.3	0.90
ML	2.0	2.5	—	2.2	0.65

39.4 Step 3: Skill Gap Analysis

Target Role: ML Engineer

Skill	Required	Current	Gap Score	Priority
Python	4	4.27	0	—
SQL	3	3.3	0	—
Git	4	4.3	0	—

Skill	Required	Current	Gap Score	Priority
ML	4	2.2	1.62	High
Deep Learning	4	0	3.20	Critical
Transformers	4	0	3.20	Critical
LLMs	4	0	3.20	Critical
RAG	4	0	3.20	Critical
Vector DBs	3	0	2.40	Critical
MLOps	3	0	2.40	Critical

39.5 Step 4: Knowledge Graph Expansion

Gaps: ML, DL, Transformers, LLMs, RAG, Vector DBs, MLOps

Prerequisite expansion:

ML ← Statistics (partially satisfied)

DL ← ML

Transformers ← DL

LLMs ← Transformers

RAG ← LLMs + Vector DBs

Vector DBs ← (none)

MLOps ← (none)

Missing foundations: Statistics (partial)

39.6 Step 5: Recommended Courses

Rank	Course	Score	Addresses
1	Statistics for ML	0.88	Statistics
2	Machine Learning Specialization	0.92	ML
3	Deep Learning Specialization	0.90	DL
4	Transformers & Attention	0.87	Transformers
5	LLM Engineering	0.85	LLMs
6	RAG Fundamentals	0.92	RAG
7	Vector Databases	0.88	Vector DBs
8	MLOps Fundamentals	0.82	MLOps

39.7 Step 6: Recommended Projects

Skill	Project	Difficulty	Duration
ML	Predict house prices	2	1 week
DL	Image classifier	3	2 weeks
Transformers	Sentiment analysis with BERT	3	2 weeks
LLMs	Build a chatbot	3	2 weeks
RAG	Document Q&A system	4	3 weeks
Vector DBs	Semantic search engine	3	2 weeks
MLOps	Deploy ML model with CI/CD	4	3 weeks

39.8 Step 7: Mentor Matching

Mentor	Expertise Overlap	Score	Reason
Dr. Sarah Chen	RAG, LLMs, Vector DBs	0.91	3 of top 5 gaps
Prof. James Lee	ML, DL	0.85	2 of top 5 gaps
Dr. Maria Garcia	MLOps, Python	0.78	1 gap + Python

Top recommendation: Dr. Sarah Chen

"Expert in RAG, LangChain, and Vector Databases — overlapping with 3 of your highest-priority skill gaps. Available 4 sessions/month. Previous mentees rated 4.8/5."

39.9 Step 8: Optimized Learning Path

Phase 1: Foundations (Weeks 1-4)

- Statistics for ML (12h)
- Project: Statistical analysis (8h)
- Assessment: Statistics

Phase 2: Core ML (Weeks 5-10)

- Machine Learning Specialization (60h)
- Project: House price prediction (10h)
- Assessment: ML

Phase 3: Deep Learning (Weeks 11-16)

- Deep Learning Specialization (80h)
- Project: Image classifier (15h)
- Assessment: DL

Phase 4: Transformers & LLMs (Weeks 17-22)

- Transformers & Attention (30h)
- LLM Engineering (40h)
- Project: Sentiment analysis (12h)
- Assessment: LLMs

Phase 5: RAG & Vector DBs (Weeks 23-28)

- RAG Fundamentals (40h)
- Vector Databases (20h)
- Project: Document Q&A system (20h)
- Assessment: RAG

Phase 6: MLOps (Weeks 29-32)

- MLOps Fundamentals (30h)
- Project: Deploy ML model (15h)
- Assessment: MLOps

Phase 7: Capstone (Weeks 33-36)

- Project: End-to-end ML system (40h)

Total: 36 weeks, ~360 hours

39.10 Step 9: Assessment & Adaptation

After Statistics assessment (score: 55%):

- Insert: "Statistics Refresher" module
- Delay ML by 1 week
- Update Statistics proficiency: 2.5 → 3.0

After ML assessment (score: 88%):

- Mark ML as satisfied
- Skip basic ML project
- Add "Advanced ML" module
- Update ML proficiency: 2.2 → 4.0

After DL assessment (score: 92%):

- Accelerate to Transformers
- Add advanced DL project
- Update DL proficiency: 0 → 4.2

39.11 Step 10: Progress Update

After 16 weeks:

Skill	Initial	Current	Target	Status
Python	4.27	4.3	4	✓
SQL	3.3	3.5	3	✓
ML	2.2	4.0	4	✓
DL	0	4.2	4	✓
Transformers	0	2.5	4	↺
LLMs	0	0	4	🕒
RAG	0	0	4	🕒

Path status: 45% complete, on track

40. Final Product Definition

40.1 Product Summary

SkillPath AI is an AI-powered learning-path recommender that performs evidence-based skill-gap analysis and generates personalized, prerequisite-aware, and continuously adaptive development paths. It combines NLP, knowledge graphs, hybrid recommendation, constraint optimization, and explainable AI to connect **skills** → **courses** → **projects** → **mentors** → **assessments** → **evidence** in a single intelligent loop.

40.2 Core Features

1. Resume & JD skill extraction (NLP)
2. Evidence-based skill profiling (multi-source)
3. Skill-gap analysis (weighted model)
4. Skill knowledge graph (Neo4j)
5. Hybrid recommendation engine (7 signals)

6. Course recommendation
7. Project recommendation
8. Mentor recommendation
9. Prerequisite-aware learning path generation
10. Learning path optimization (OR-Tools)
11. Adaptive learning (assessment-driven)
12. Assessment engine
13. GitHub skill verification
14. Explainable AI layer
15. AI learning assistant (RAG)
16. What-If career simulation
17. Skill digital twin

40.3 Key Innovations

1. **Evidence-weighted skill confidence model**
2. **Prerequisite-aware learning path optimization**
3. **7-signal hybrid recommendation framework**
4. **Deterministic explainability layer**
5. **Adaptive path re-optimization**
6. **Skill digital twin**

40.4 Technical Architecture

- **Frontend:** React + TypeScript + Tailwind
- **Backend:** FastAPI + Python
- **Databases:** PostgreSQL + Neo4j + pgvector + Redis
- **ML:** spaCy + Sentence Transformers + Scikit-learn
- **Optimization:** OR-Tools
- **LLM:** LangChain + OpenAI/Gemini
- **Deployment:** Docker + GitHub Actions

40.5 Research Contribution

- Novel evidence-weighted skill confidence model
- Constrained optimization formulation for learning paths

- Hybrid recommendation framework for competency development
- Explainable recommendation taxonomy for EdTech
- Adaptive learning path re-optimization
- Skill digital twin concept

40.6 MVP Scope

- Employee profile
- Resume skill extraction
- Skill taxonomy
- Role definitions
- Basic gap analysis
- Content-based course recommendation
- Topological path generation
- Basic UI

40.7 Advanced Scope

- Evidence-based profiling
- Knowledge graph
- Hybrid recommendation
- Path optimization
- Projects + mentors
- Assessments
- Adaptive learning
- Explainability
- AI assistant
- GitHub integration
- What-If simulation
- Digital twin
- Full evaluation

40.8 Success Metrics

Metric	Target
Skill extraction F1	≥ 0.85

Metric	Target
Gap ranking NDCG@10	≥ 0.80
Recommendation Precision@10	≥ 0.70
Prerequisite violations	0
Path generation time	$< 3s$
User satisfaction (SUS)	≥ 75
Explainability satisfaction	$\geq 80\%$

40.9 Future Scope

- Real-time collaboration
- Mobile app
- VR/AR learning
- Blockchain credentials
- Federated learning
- Multi-language
- Advanced IRT
- Reinforcement learning
- Emotion detection
- Team-level optimization

41. What Makes This Project Unique?

The 10 Strongest Technical/Research Differentiators

1. Evidence-Based Skill Profiling with Confidence Scores

What: Aggregates multi-source evidence (resume, GitHub, assessments, projects, courses, certifications) into a single proficiency + confidence score using a novel formula that accounts for reliability, strength, and recency.

Why unique: Conventional platforms rely on self-reported skills or single-source data. SkillPath AI quantifies *how sure* it is about each skill, handles conflicting evidence, and flags low-confidence inferences. This is a genuine research contribution — a formal evidence aggregation model for competency profiling.

2. Prerequisite-Aware Learning Path Generation

What: Constructs an ordered learning path by traversing a skill prerequisite DAG, inserting missing foundations, and ensuring no skill is learned before its prerequisites.

Why unique: Existing recommenders suggest independent courses. SkillPath AI models *dependencies between skills* and generates a *sequence*, not a set. This transforms recommendation into curriculum design.

3. Learning Path Optimization via Constraint Programming

What: Formulates path generation as a constrained optimization problem (OR-Tools CP-SAT) with objectives (minimize time, maximize coverage) and constraints (prerequisites, availability, deadline).

Why unique: Most EdTech systems use greedy heuristics. SkillPath AI uses formal optimization, producing provably better paths. This is a novel application of OR to learning-path design.

4. 7-Signal Hybrid Recommendation Engine

What: Combines semantic similarity, skill-gap relevance, prerequisite fit, difficulty fit, user preference, collaborative filtering, and knowledge-graph reasoning into a single weighted score.

Why unique: Most systems use 1–2 signals. SkillPath AI's 7-signal hybrid is comprehensive, tunable, and each signal is independently explainable. The ablation study will quantify each signal's contribution.

5. Deterministic Explainability Layer

What: Every recommendation has a deterministic, evidence-grounded explanation generated from the scoring components — not from an LLM. The LLM only *verbalizes* the explanation.

Why unique: This solves the LLM hallucination problem in explainable AI. The explanation is *derived from the algorithm*, not *generated by a language model*. This is a methodological contribution to explainable recommendation.

6. Adaptive Learning Path Re-Optimization

What: The learning path dynamically re-optimizes based on assessment results, course completions, and user feedback — inserting remediation or skipping mastered content.

Why unique: Static paths are the norm. SkillPath AI treats the path as a *living artifact* that responds to evidence. This closes the loop between learning and assessment.

7. Knowledge Graph as the Backbone

What: A Neo4j knowledge graph encodes skills, prerequisites, resources, mentors, and roles as nodes and relationships — enabling graph traversal for prerequisite reasoning, mentor matching, and path finding.

Why unique: Most recommenders use flat tables. SkillPath AI's graph-native approach enables *multi-hop reasoning* (e.g., "find mentors who are experts in skills that are prerequisites of my gaps"). This is a structural innovation.

8. Integration of Courses + Projects + Mentors + Assessments

What: The system doesn't just recommend courses. It connects every skill gap to a *bundle*: a course to learn, a project to practice, a mentor to guide, and an assessment to verify.

Why unique: Traditional platforms are course-centric. SkillPath AI is *competency-centric* — courses are just one input. This holistic view is closer to how skills are actually built.

9. Skill Digital Twin

What: A dynamic, visual representation of the employee's skill profile showing current level, required level, gap, confidence, evidence, and progress over time.

Why unique: This is a novel UI/UX concept — a *digital twin* of competence. It makes the abstract concept of "skill profile" tangible and actionable. No existing platform offers this.

10. Research-Grade Evaluation Framework

What: A comprehensive evaluation framework comparing 5 recommendation approaches (popularity, content, semantic, KG, hybrid) with metrics for recommendations, skill gaps, learning paths, and user satisfaction.

Why unique: Most student projects lack rigorous evaluation. SkillPath AI is designed from the ground up to be *evaluable* — with ablation studies, offline metrics, and online experiments. This makes it publishable.

Summary Table

#	Differentiator	Type	Why It Matters
1	Evidence-based profiling	Research	Quantifies confidence; handles conflict
2	Prerequisite-aware paths	Technical	Generates sequences, not sets
3	Constraint optimization	Research	Formal optimization vs. heuristics
4	7-signal hybrid	Technical	Comprehensive, tunable, explainable
5	Deterministic explainability	Research	Solves LLM hallucination
6	Adaptive re-optimization	Technical	Living paths, not static
7	Knowledge graph backbone	Technical	Multi-hop reasoning
8	Courses + Projects + Mentors	Product	Competency-centric, not course-centric
9	Skill Digital Twin	Product	Novel visualization
10	Research-grade evaluation	Research	Publishable results

End of PRD

This document is implementation-ready and suitable for both a college project and a potential research paper. The development team can begin implementation immediately, starting with Phase 1 (Foundation) and progressing through the roadmap. The MVP provides a working demonstration, while the Advanced Version delivers the full research contribution.